UNIVERSITATEA [NUMELE UNIVERSITĂȚII]

Facultatea de [Facultate]

Departamentul de [Departament]

Programul de master: [Denumire program]

Sistem E-Medicine pentru Analizarea și Monitorizarea Datelor Epidemiologice bazat pe Modelul Matematic SIR

LUCRARE DE DISERTAȚIE

Coordonator științific:

Prof. Dr. [Nume Coordonator]

[Titlul / Funcția]

Absolvent:

[Prenume Nume]

Seria [an], Grupa [grupă]

[Orașul], 2026

Rezumat

Lucrarea de față prezintă proiectarea și realizarea unui sistem e-medicine dedicat analizării și monitorizării datelor epidemiologice, având ca fundament matematic modelul SIR (Susceptibili–Infectați–Recuperați). Modelul este implementat și testat comparativ în patru limbaje de programare cu utilizare intensă în calculul științific: R, Julia, Python și SAS.

Implementările sunt evaluate pe mai multe configurații hardware, urmărindu-se comportamentul algoritmilor din perspectiva performanței și a scalabilității în funcție de dimensiunea seturilor de date procesate. Datele epidemiologice utilizate sunt stocate într-o bază de date distribuită YugabyteDB, rulată într-un cluster Kubernetes, asigurând disponibilitate ridicată, toleranță la erori și distribuție eficientă a datelor.

Pentru centralizarea, compararea și vizualizarea rezultatelor este dezvoltată o aplicație web dedicată, compusă dintr-un API backend implementat în limbajul Go și o interfață frontend realizată în Vue.js. Aceasta oferă o perspectivă unificată asupra performanței și comportamentului modelelor epidemiologice analizate, permițând consultarea rezultatelor rulărilor, a seturilor de date utilizate și a graficelor asociate.

Cuvinte cheie: model SIR, epidemiologie computațională, e-medicine, calcul comparativ, YugabyteDB, Kubernetes, Go, Vue.js, benchmark de performanță.

Cuprins

Rezumat **i**

Listă de figuri **vii**

Listă de tabele **viii**

Lista algoritmilor **ix**

1 Introducere **1**

1.1 Context și motivație 1
1.2 Obiectivele lucrării 2
1.3 Contribuții originale 2
1.4 Structura lucrării 3

2 Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase **4**

2.1 Fundamente de epidemiologie 4
2.1.1 Definiții și concepte fundamentale 4
2.1.2 Parametrii fundamentali ai transmiterii 5
2.2 Modele compartimentale în epidemiologie 5
2.2.1 Cadrul general 5
2.2.2 Modelul SI 6
2.2.3 Modelul SIS 6
2.2.4 Modelul SEIR 7
2.2.5 Modelul SIRS 7
2.3 Modelul SIR — Analiză Matematică Detaliată 7
2.3.1 Ipoteze și derivarea ecuațiilor din principii prime 7
2.3.2 Proprietăți matematice fundamentale 8
2.3.3 Analiza echilibrelor și stabilitate 10
2.3.4 Numărul de reproducere de bază $\mathcal{R}_0$ 11
2.3.5 Pragul de imunizare în turmă (herd immunity) 12
2.3.6 Ecuația dimensiunii finale a epidemiei 13

2.3.7 Extensii ale modelului SIR 14
2.4 Metode Numerice pentru Ecuații Diferențiale Ordinare 15
2.4.1 Clasificarea metodelor numerice 15
2.4.2 Metoda Euler explicită 15
2.4.3 Metoda Runge-Kutta de ordinul 4 (RK4) 16
2.4.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince) . . . 17
2.4.5 Stabilitate numerică și probleme stiff 19
2.5 Date Epidemiologice — Surse și Preprocesare 19
2.5.1 Surse de date epidemiologice 19
2.5.2 Structura datelor și preprocesare 20
2.5.3 Estimarea parametrilor din date reale 20

3 Tehnologii și Instrumente Utilizate 21
3.1 Limbaje de programare pentru calcul științific 21
3.1.1 R 21
3.1.2 Julia 22
3.1.3 Python 23
3.1.4 SAS 25
3.1.5 Analiză comparativă a limbajelor 26
3.2 Baze de date distribuite — YugabyteDB 26
3.2.1 Concepte de baze de date distribuite 26
3.2.2 Arhitectura YugabyteDB 27
3.2.3 Sharding și replicare 27
3.3 Orchestrarea containerelor — Kubernetes 28
3.3.1 Containere și nevoia de orchestrare 28
3.3.2 Arhitectura Kubernetes 28
3.3.3 Resurse Kubernetes relevante 29
3.4 Backend API — Go 29
3.4.1 Caracteristici și filosofia limbajului Go 29
3.4.2 Ecosistem pentru API-uri REST 30
3.5 Frontend — Vue.js 30
3.5.1 Arhitectura Vue 3 și Composition API 30
3.5.2 Ecosistem și biblioteci utilizate 30

4 Arhitectura Sistemului E-Medicine 32
4.1 Vedere de ansamblu 32
4.2 Cerințe funcționale și non-funcționale 33
4.2.1 Cerințe funcționale 33
4.2.2 Cerințe non-funcționale 34
4.3 Arhitectura bazei de date distribuite 35

4.3.1 Schema relațională 35
4.3.2 Modelul datelor epidemiologice 36
4.3.3 Strategia de partiționare și replicare în YugabyteDB 36
4.4 Arhitectura API-ului backend Go 37
4.4.1 Structura modulară a proiectului 37
4.4.2 Mecanismul de autentificare JWT 38
4.4.3 Diagrama endpointurilor REST 39
4.4.4 Integrarea cu runner-ele de algoritmi 40
4.5 Arhitectura frontend Vue.js 40
4.5.1 Structura componentelor 40
4.5.2 Fluxul de date și state management 41
4.6 Infrastructura Kubernetes 42
4.6.1 Topologia cluster-ului 42
4.6.2 Deployment YugabyteDB pe Kubernetes 43
4.6.3 Deployment API Go și frontend Vue.js 43

5 Implementarea Modelului SIR 44
5.1 Seturile de date utilizate 44
5.1.1 Sursa datelor și formatul brut 44
5.1.2 Preprocesarea și conversia în formatul sistemului 45
5.1.3 Parametrii estimați și scenariile de simulare 45
5.2 Implementarea în R 46
5.2.1 Structura proiectului și dependențe 46
5.2.2 Definiția sistemului ODE și metoda RK4 46
5.2.3 Colectarea metricilor și stocarea în baza de date 47
5.3 Implementarea în Julia 48
5.3.1 Structura proiectului și pachete 48
5.3.2 ODEProblem și solver-ul Tsit5 48
5.3.3 Colectarea metricilor și inserarea bulk 49
5.4 Implementarea în Python 50
5.4.1 Structura proiectului și dependențe 50
5.4.2 scipy.integrate.solve_ivp cu metoda RK45 50
5.4.3 Colectarea metricilor și inserarea bulk 51
5.5 Implementarea de referință în SAS/IML 52
5.5.1 RK4 manual în PROC IML 52
5.5.2 Considerații privind precizia și limitările SAS/IML 53
5.6 Interfața comună de integrare cu API-ul Go 53
5.6.1 Protocolul de comunicare prin cozi Redis 53
5.6.2 Schema mesajului de job 54

5.6.3 Bucla de consum și tratarea erorilor 54

6 Evaluarea Performanței și Scalabilității 56
6.1 Metodologia de benchmarking 56
6.1.1 Configurații hardware utilizate 56
6.1.2 Metrici urmărite 56
6.1.3 Protocoale de testare și reproductibilitate 56
6.2 Scalabilitate în funcție de dimensiunea setului de date 56
6.3 Comparație timp de execuție — R vs Julia vs Python vs SAS 57
6.4 Consum de memorie și resurse 57
6.5 Acuratețea numerică a rezultatelor 57
6.6 Performanța bazei de date distribuite 57
6.6.1 Latență la scriere și citire 57
6.6.2 Comportament sub load — teste de stress 57
6.7 Performanța API-ului Go 57
6.8 Analiză comparativă sintetică 58

7 Aplicația Web E-Medicine 59
7.1 Descrierea funcționalităților aplicației 59
7.2 Backend Go — Implementare detaliată 59
7.2.1 Structura proiectului Go 59
7.2.2 Implementarea endpointurilor REST 59
7.2.3 Gestionarea execuției algoritmilor 59
7.2.4 Conexiunea la YugabyteDB 59
7.2.5 Autentificare și securitate 60
7.3 Frontend Vue.js — Implementare detaliată 60
7.3.1 Structura componentelor și pagini 60
7.3.2 Dashboard epidemiologic 60
7.3.3 Vizualizarea graficelor SIR 60
7.3.4 Comparatorul de rezultate între limbaje 60
7.3.5 Interfața pentru seturile de date 60
7.4 Fluxuri de utilizare (use cases) 60
7.5 Testare și validare funcțională 61

8 Concluzii și Direcții de Dezvoltare Viitoare 62
8.1 Sinteza contribuțiilor 62
8.2 Concluzii privind implementările modelului SIR 62
8.3 Concluzii privind infrastructura distribuită 62
8.4 Limitări ale lucrării 62
8.5 Direcții de dezvoltare viitoare 63

Bibliografie **64**

A Cod Sursă — Implementarea R **67**

B Cod Sursă — Implementarea Julia **68**

C Cod Sursă — Implementarea Python **69**

D Cod Sursă — Implementarea SAS **70**

E Schema Bazei de Date **71**

F Configurații Kubernetes **72**

G Documentația API REST **73**

Listă de figuri

2.1 Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare. . . . 8

4.1 Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către SPA sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB. 33

4.2 Diagrama entitate-relație a bazei de date. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*; raportul de cardinalitate 1 : N este indicat pe fiecare relație. 35

4.3 Topologia cluster-ului Kubernetes. Ingressul este expus ca LoadBalancer Service în fața clusterului. Worker Node 1 găzduiește frontend-ul, primul runner și prima instanță YugabyteDB tserver; Worker Node 2 găzduiește celelalte două runner-e și doi tserveri. 42

5.1 Ciclul de viață al unui job de simulare: tranziția de stare în `simulation_runs` în funcție de evenimentele din runner. 54

Listă de tabele

3.1 Comparație a limbajelor de programare utilizate 26

4.1 Structura minimă a fișierului CSV de date epidemiologice. 36
4.2 Endpointurile REST ale API-ului backend. 39

5.1 Coloanele OWID utilizate la construirea seturilor de date SIR. 44
5.2 Parametrii SIR estimați pentru scenariile de simulare. 45

Lista algoritmilor

2.1 Metoda Euler pentru sistemul SIR 16
2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR 17

Capitolul 1

Introducere

1.1 Context și motivație

Bolile infecțioase reprezintă una dintre cele mai vechi și persistente provocări ale umanității. De la pandemia de gripă din 1918 la epidemia de SARS din 2003 și, mai recent, pandemia de COVID-19 declanșată în 2019, evenimentele epidemice de amploare au demonstrat în mod repetat capacitatea lor de a perturba profund structurile sociale, economice și sanitare ale societăților moderne [1, 2].

Înțelegerea și controlul dinamicii bolilor infecțioase nu constituie doar o problemă medicală, ci reprezintă o provocare interdisciplinară care îmbină epidemiologia, matematica aplicată, informatica și știința datelor. La baza acestei înțelegeri stau *modelele matematice compartimentale*, instrumente formale care permit cuantificarea proceselor de transmitere, predicția evoluției epidemiilor și evaluarea intervențiilor de sănătate publică.

Modelul **SIR** (Susceptibili-Infectați-Recuperați), introdus de Kermack și McKendrick în lucrarea fondatoare din 1927 [3], rămâne și astăzi fundamentul teoretic al epidemiologiei matematice. Simplitatea sa elegantă — trei ecuații diferențiale ordinare cuplate — ascunde o bogăție matematică remarcabilă și o putere predictivă validată de zeci de ani de aplicații empirice.

Odată cu creșterea exponențială a volumului de date epidemiologice disponibile și cu diversificarea instrumentelor computaționale moderne, apare necesitatea unor *sisteme informatice integrate* capabile să gestioneze atât stocarea distribuită a datelor, cât și rularea eficientă a modelelor matematice și vizualizarea rezultatelor. Alegerea limbajului de programare și a arhitecturii software are un impact direct și semnificativ asupra performanței, scalabilității și mentenabilității unui astfel de sistem.

Lucrarea de față se înscrie în această direcție, propunând proiectarea și realizarea unui sistem e-medicine pentru analizarea și monitorizarea datelor epidemiologice, cu accent pe implementarea și analiza comparativă a modelului SIR în patru medii de

programare reprezentative pentru spectrul aplicațiilor de calcul științific: **R**, **Julia**, **Python** și **SAS**.

1.2 Obiectivele lucrării

Prezenta lucrare urmărește realizarea unui sistem coerent și cuprinzător, structurat în jurul mai multor obiective principale:

1. **Fundamentarea matematică riguroasă** a modelului SIR prin derivarea ecuațiilor din principii prime, demonstrarea proprietăților matematice fundamentale (existență, unicitate, pozitivitate, conservare) și analiza calitativă a soluțiilor (echilibre, stabilitate, numărul de reproducere de bază $\mathcal{R}_0$).

2. **Implementarea modelului SIR** în patru limbaje de programare — R, Julia, Python și SAS — utilizând metodele numerice adecvate pentru rezolvarea sistemelor de ecuații diferențiale ordinare (metoda Euler, Runge-Kutta de ordinul 4, metode adaptive).

3. **Evaluarea comparativă a performanței** implementărilor pe multiple configurații hardware, urmărind metricile de timp de execuție, consum de memorie și scalabilitate în funcție de dimensiunea setului de date.

4. **Proiectarea și implementarea infrastructurii distribuite** pentru stocarea datelor epidemiologice, bazată pe baza de date YugabyteDB rulată într-un cluster Kubernetes, asigurând disponibilitate ridicată și distribuție eficientă a datelor.

5. **Dezvoltarea aplicației web e-medicine**, compusă dintr-un API backend implementat în Go și o interfață frontend Vue.js, care integrează toate componentele sistemului și oferă o perspectivă unificată asupra rezultatelor.

1.3 Contribuții originale

Contribuțiile originale ale acestei lucrări pot fi sintetizate după cum urmează:

- **Analiză comparativă sistematică** a implementărilor modelului SIR în R, Julia, Python și SAS, acoperind atât corectitudinea numerică, cât și performanța computațională pe configurații hardware diverse.

- **Metodologie de benchmarking reproducibilă** pentru evaluarea performanței algoritmilor epidemiologici în limbaje de programare științifică, inclusiv protocoale de testare și metrici standardizate.

- **Arhitectură distribuită de stocare a datelor epidemiologice** bazată pe YugabyteDB și Kubernetes, cu schema relațională optimizată pentru interogări specifice domeniului epidemiologic.
- **Sistem web integrat** pentru vizualizarea și compararea rezultatelor modelării epidemiologice, combinând un API REST performant (Go) cu o interfață web modernă și reactivă (Vue.js).

1.4 Structura lucrării

Lucrarea este organizată în opt capitole, după cum urmează:

Capitolul 2 (*Epidemiologie Computațională și Modelarea Matematică*) prezintă fundamentele epidemiologiei matematice. Sunt introduse modelele compartimentale (SI, SIS, SIR, SEIR, SIRS), cu accent pe analiza matematică riguroasă a modelului SIR: derivare, teoreme de existență și unicitate, analiza stabilității, numărul de reproducere de bază $\mathcal{R}_0$ și ecuația dimensiunii finale. Capitolul include și o trecere în revistă a metodelor numerice utilizate pentru rezolvarea sistemelor de ecuații diferențiale ordinare.

Capitolul 3 (*Tehnologii și Instrumente Utilizate*) prezintă fundamentele teoretice și caracteristicile tehnice ale instrumentelor utilizate: limbajele de programare R, Julia, Python și SAS, baza de date distribuită YugabyteDB, sistemul de orchestrare Kubernetes, limbajul de programare Go și framework-ul frontend Vue.js.

Capitolul 4 (*Arhitectura Sistemului E-Medicine*) descrie arhitectura globală a sistemului, cerințele funcționale și non-funcționale, schema bazei de date distribuite, arhitectura API-ului backend și structura componentelor frontend.

Capitolul 5 (*Implementarea Modelului SIR*) detaliază implementările practice ale modelului SIR în fiecare limbaj de programare, inclusiv seturile de date utilizate, parametrii modelului și interfața de integrare cu API-ul Go.

Capitolul 6 (*Evaluarea Performanței și Scalabilității*) prezintă metodologia de benchmarking, configurațiile hardware utilizate și rezultatele comparative ale evaluărilor de performanță.

Capitolul 7 (*Aplicația Web E-Medicine*) detaliază implementarea aplicației web, documentând atât backend-ul Go, cât și frontend-ul Vue.js, cu descrierea funcționalităților și a fluxurilor de utilizare.

Capitolul 8 (*Concluzii și Direcții Viitoare*) sintetizează contribuțiile lucrării, formulează concluzii privind implementările și infrastructura, și identifică direcții de dezvoltare ulterioară.

Lucrarea se încheie cu **Bibliografie** și **Anexe** care conțin codul sursă complet al implementărilor, schema bazei de date, configurațiile Kubernetes și documentația API-ului REST.

Capitolul 2

Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase

2.1 Fundamente de epidemiologie

2.1.1 Definiții și concepte fundamentale

Epidemiologia este știința care studiază distribuția, determinanții și controlul stărilor sau evenimentelor legate de sănătate în populații definite [1]. În sens mai specific, *epidemiologia infecțioasă* se ocupă cu dinamica transmiterii bolilor cauzate de agenți patogeni biologici (virusuri, bacterii, paraziți) în populații umane sau animale.

Definiție 2.1 (Epidemic, pandemic, endemic)**.** O boală infecțioasă se numește **endemică** dacă persistă la nivel constant într-o regiune geografică definită. Se numește **epidemie** dacă numărul de cazuri depășește în mod semnificativ nivelul endemic așteptat, și **pandemie** dacă afectează mai multe continente sau o mare parte a globului.

Definiție 2.2 (Incidență și prevalență)**. Incidența** reprezintă numărul de cazuri noi ale unei boli apărute într-o populație de risc, într-un interval de timp specificat. Se exprimă ca rată:

$$\text{Incidență} = \frac{\text{cazuri noi în intervalul } [t_1, t_2]}{\text{populație la risc} \times (t_2 - t_1)}. \tag{2.1}$$

Prevalența reprezintă proporția indivizilor afectați la un moment dat:

$$\text{Prevalență} = \frac{\text{cazuri existente la momentul } t}{\text{populație totală la momentul } t}. \tag{2.2}$$

2.1.2 Parametrii fundamentali ai transmiterii

Definiție 2.3 (Intervalul serial și intervalul de generație)**. Intervalul serial** este durata medie dintre apariția simptomelor la cazul primar și la cazul secundar generat de acesta. **Intervalul de generație** este durata medie dintre infecția cazului primar și infecția unui caz secundar.

Definiție 2.4 (Perioada de incubație)**. Perioada de incubație** este intervalul de timp dintre momentul infectării și apariția primelor simptome. Este distinctă de *perioada de latență*, care reprezintă intervalul dintre infecție și debutul infecțiozității.

Definiție 2.5 (Numărul de reproducere de bază $\mathcal{R}_0$)**.** Numărul de reproducere de bază $\mathcal{R}_0$ este definit ca numărul mediu de infecții secundare produse de un singur individ infectat introdus într-o populație complet susceptibilă, pe durata perioadei sale infecțioase [4]. Este cel mai important parametru al epidemiologiei matematice, deoarece determină potențialul epidemic al unui agent patogen.

O consecință imediată a Teoremă 2.5 este că:

- dacă $\mathcal{R}_0 < 1$, fiecare caz infectat generează în medie mai puțin de un caz nou, astfel că epidemia se stinge spontan;
- dacă $\mathcal{R}_0 > 1$, fiecare caz generează în medie mai mult de un caz nou, epidemia se propagă exponențial în stadiul inițial;
- dacă $\mathcal{R}_0 = 1$, epidemia se menține la nivel endemic instabil.

Numărul de reproducere *efectiv* (sau în timp real) $\mathcal{R}(t)$ generalizează $\mathcal{R}_0$ pentru o populație parțial imunizată:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N}, \tag{2.3}$$

unde $S(t)$ este numărul de indivizi susceptibili la momentul t și N este mărimea populației totale.

2.2 Modele compartimentale în epidemiologie

2.2.1 Cadrul general

Modelele compartimentale reprezintă clasa fundamentală de modele matematice în epidemiologie [2]. Principiul lor constă în împărțirea populației în *compartimente* care descriu stări epidemiologice distincte, iar tranziția dintre compartimente este guvernată de un sistem de ecuații diferențiale ordinare (EDO).

Fie N mărimea totală a populației (presupusă constantă). Fiecare compartiment $X_i(t)$ reprezintă numărul de indivizi în starea i la momentul t, cu $\sum_i X_i(t) = N$. Dinamica tranzițiilor este descrisă de sistemul:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t, \boldsymbol{\theta}), \quad \mathbf{x}(0) = \mathbf{x}_0, \tag{2.4}$$

unde $\mathbf{x} = (X_1, \ldots, X_n)^\intercal$, $\boldsymbol{\theta}$ este vectorul parametrilor și $\mathbf{f}$ descrie fluxurile nete dintre compartimente.

2.2.2 Modelul SI

Cel mai simplu model compartimental este modelul **SI** (Susceptibili-Infectați), care presupune că infecția este permanentă (nu există recuperare):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N}. \end{cases} \tag{2.5}$$

Cu condiția $S + I = N$, sistemul (2.5) se reduce la o ecuație logistică în I, cu soluția analitică:

$$I(t) = \frac{N}{1 + \left(\frac{N - I_0}{I_0}\right) e^{-\beta t}}, \tag{2.6}$$

unde $I_0 = I(0)$. Toți indivizii sunt infectați pe termen lung ($I(\infty) = N$).

2.2.3 Modelul SIS

Modelul **SIS** (Susceptibili-Infectați-Susceptibili) extinde modelul SI prin includerea recuperării fără imunitate, permițând re-infectarea:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \gamma I, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.7}$$

Parametrul $\gamma > 0$ reprezintă rata de recuperare ($1/\gamma$ este durata medie a perioadei infecțioase). Pentru modelul SIS, $\mathcal{R}_0 = \beta/\gamma$, iar:

- dacă $\mathcal{R}_0 \leq 1$: echilibrul fără boală $I^* = 0$ este stabil global;
- dacă $\mathcal{R}_0 > 1$: există un echilibru endemic stabil $I^* = N(1 - 1/\mathcal{R}_0) > 0$.

2.2.4 Modelul SEIR

Modelul **SEIR** introduce compartimentul E (Expuși / în perioadă de latență), care ține cont de faptul că indivizii infectați nu sunt imediat infecțioși:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dE}{dt} = \frac{\beta SI}{N} - \sigma E, \\ \frac{dI}{dt} = \sigma E - \gamma I, \\ \frac{dR}{dt} = \gamma I, \end{cases} \tag{2.8}$$

unde $\sigma = 1/T_E$ este rata de tranziție din starea expusă în starea infecțioasă (T_E = durata medie de incubație). Numărul de reproducere de bază pentru modelul SEIR este $\mathcal{R}_0 = \beta/\gamma$, identic cu cel al modelului SIR.

2.2.5 Modelul SIRS

Modelul **SIRS** generalizează modelul SIR prin imunitate temporară (indivizii recuperați pot deveni din nou susceptibili după un interval de timp):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \xi R, \\ \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} = \gamma I - \xi R, \end{cases} \tag{2.9}$$

unde $\xi = 1/T_R$ este rata de pierdere a imunității (T_R = durata medie a imunității). Dacă $\mathcal{R}_0 > 1$, modelul SIRS admite un echilibru endemic stabil.

2.3 Modelul SIR — Analiză Matematică Detaliată

2.3.1 Ipoteze și derivarea ecuațiilor din principii prime

Modelul SIR, formulat de Kermack și McKendrick [3], se bazează pe o serie de ipoteze simplificatoare care trebuie înțelese explicit:

1. **Populație închisă și omogenă:** Nu există nașteri, decese naturale sau migrații; $N = \text{const}$. Fiecare individ are aceeași probabilitate de a intra în contact cu orice alt individ (amestecare omogenă).

2. **Transmitere prin acțiune de masă:** Rata de contact eficient (producătoare de infecție) este proporțională cu produsul $S \cdot I$.

3. **Imunitate permanentă:** Un individ recuperat nu poate fi re-infectat; compartimentul R este absorbant.

4. **Durata infecțioasă exponențial distribuită:** Rata de recuperare γ este constantă, iar durata medie a perioadei infecțioase este $1/\gamma$.

5. **Fără mortalitate indusă de boală:** Sau, echivalent, mortalitatea specifică bolii este neglijabilă față de dinamica transmiterii.

Derivarea termenului de infecție. Considerăm că în unitatea de timp, un individ susceptibil are în medie c contacte cu alți indivizi aleși uniform din populație. Probabilitatea ca un contact arbitrar să fie cu un individ infectat este I/N, iar probabilitatea de transmitere la contact eficient este p. Astfel, rata de infecție a unui susceptibil este $c \cdot p \cdot I/N = \beta I/N$, unde $\beta = cp$ este parametrul de transmitere. Fluxul total de noi infecții pe unitate de timp este $(\beta I/N) \cdot S$.

Sistemul de ecuații diferențiale ordinare al modelului SIR este:

$$\boxed{\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned}} \tag{2.10}$$

cu condiții inițiale $S(0) = S_0 > 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$ și $S_0 + I_0 + R_0 = N$.

Diagrama de flux a compartimentelor este prezentată în Fig. 2.1.

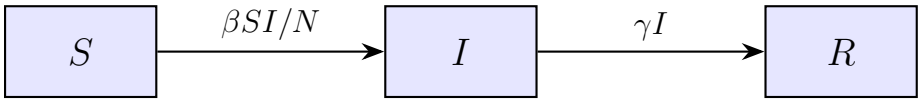


Figura 2.1: Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare.

2.3.2 Proprietăți matematice fundamentale

Înainte de a analiza comportamentul calitativ al soluțiilor, stabilim proprietățile matematice esențiale: existența și unicitatea soluției, conservarea populației totale și invarianța domeniului pozitiv.

Teoremă 2.6 (Existență și unicitate globală)**.** *Fie $\beta, \gamma > 0$, $N > 0$ și $(S_0, I_0, R_0) \in \mathbb{R}^3$ cu $S_0, I_0 > 0$, $R_0 \geq 0$. Atunci sistemul* (2.10) *admite o unică soluție globală $(S, I, R) \in C^1([0, \infty); \mathbb{R}^3)$.*

Demonstrație. Definim $\mathbf{x} = (S, I, R)^\intercal$ și $\mathbf{f}(\mathbf{x}) = (-\beta SI/N,\ \beta SI/N - \gamma I,\ \gamma I)^\intercal$. Funcția $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^3$ este de clasă C^∞ (polinomul de gradul doi în S și I este neted), în particular continuu diferențiabilă. Prin *teorema Picard-Lindelöf* (Cauchy-Lipschitz), există o soluție locală unică pe un interval $[0, T_{\max})$.

Existența *globală* ($T_{\max} = \infty$) se stabilește prin mărginirea soluției. Deoarece $S(t), I(t), R(t) \geq 0$ (Teorema 2.8) și $S + I + R = N$ (Teorema 2.7), rezultă că $\|\mathbf{x}(t)\| \leq N\sqrt{3}$ pentru orice $t \in [0, T_{\max})$. Soluția mărginită nu poate prezenta blow-up în timp finit, deci $T_{\max} = \infty$. □

Teoremă 2.7 (Conservarea populației totale)**.** *Pentru orice soluție a sistemului* (2.10)*,*

$$S(t) + I(t) + R(t) = N \quad \forall\, t \geq 0. \tag{2.11}$$

Demonstrație. Calculăm derivata sumei:

$$\begin{aligned} \frac{d}{dt}(S + I + R) &= \frac{dS}{dt} + \frac{dI}{dt} + \frac{dR}{dt} \\ &= -\frac{\beta SI}{N} + \left(\frac{\beta SI}{N} - \gamma I\right) + \gamma I \\ &= 0. \end{aligned}$$

Prin integrare, $S(t) + I(t) + R(t) = S_0 + I_0 + R_0 = N$. □

Teoremă 2.8 (Pozitivitate și mărginire)**.** *Dacă $S_0 > 0$, $I_0 > 0$, $R_0 \geq 0$, atunci:*

$$S(t) > 0, \quad I(t) > 0, \quad R(t) \geq 0, \quad \forall\, t > 0. \tag{2.12}$$

În particular, $S(t), I(t), R(t) \in [0, N]$ pentru orice $t \geq 0$.

Demonstrație. **(a) Pozitivitatea lui S:** Din ecuația $dS/dt = -(\beta I/N)S$, aceasta este o ecuație diferențială liniară omogenă în S cu coeficient $-\beta I(t)/N$. Soluția este:

$$S(t) = S_0 \cdot \exp\left(-\frac{\beta}{N}\int_0^t I(s)\, ds\right). \tag{2.13}$$

Deoarece integrala este finită și $S_0 > 0$, rezultă $S(t) > 0$ pentru orice $t \geq 0$ (presupunând $I(s) \geq 0$, demonstrat mai jos).

(b) Pozitivitatea lui I: Rescrierea ecuației a doua ca $dI/dt = (\beta S(t)/N - \gamma)\, I$

arată că aceasta este liniară în I:

$$I(t) = I_0 \cdot \exp\left(\int_0^t \left[\frac{\beta S(\tau)}{N} - \gamma\right] d\tau\right) > 0, \tag{2.14}$$

deoarece $I_0 > 0$ și exponențiala este strict pozitivă.

(c) Non-negativitatea lui R: Din $dR/dt = \gamma I(t) > 0$ și $R_0 \geq 0$, funcția R este strict crescătoare, deci $R(t) > R_0 \geq 0$.

Mărginirea urmează din Teorema 2.7: $S(t) \leq N$, $I(t) \leq N$, $R(t) \leq N$. □

Observație 2.9. Teorema 2.8 implică că domeniul pozitiv

$$\Omega = \{(S, I, R) \in \mathbb{R}^3_{\geq 0} : S + I + R = N\} \tag{2.15}$$

este *invariant pozitiv* față de fluxul sistemului (2.10). Prin urmare, este suficient să studiem comportamentul soluțiilor pe Ω, care este un simplex în $\mathbb{R}^3$.

Teoremă 2.10 (Monotonia compartimentelor)**.** *De-a lungul oricărei soluții a sistemului* (2.10)*:*

(i) $S(t)$ *este strict descrescătoare;*

(ii) $R(t)$ *este strict crescătoare;*

(iii) $I(t)$ *atinge un singur maxim în* $t^* = \inf\{t > 0 : S(t) = \gamma N/\beta\}$ *(dacă* $S_0 > \gamma N/\beta$*); altfel este descrescătoare.*

Demonstraţie. (i) $dS/dt = -\beta SI/N < 0$ deoarece $S > 0$, $I > 0$. *(ii)* $dR/dt = \gamma I > 0$. *(iii)* $dI/dt = (\beta S/N - \gamma)I > 0 \Leftrightarrow S > \gamma N/\beta$. Deoarece S este strict descrescătoare, există cel mult un moment t^* când $S(t^*) = \gamma N/\beta$, care este maximul lui I. □

2.3.3 Analiza echilibrelor și stabilitate

Punctele de echilibru ale sistemului (2.10) sunt obținute din condiția $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Ecuațiile conduc la:

$$-\frac{\beta S^* I^*}{N} = 0, \quad \frac{\beta S^* I^*}{N} - \gamma I^* = 0, \quad \gamma I^* = 0. \tag{2.16}$$

Din a treia ecuație, $I^* = 0$. Substituind, obținem că orice punct de forma $(S^*, 0, N - S^*)$ cu $S^* \in [0, N]$ este echilibru. Mulțimea echilibrelor este:

$$\mathcal{E} = \{(S^*, 0, N - S^*) : S^* \in [0, N]\} \subset \Omega. \tag{2.17}$$

Observație 2.11. Spre deosebire de modelele SIS și SIRS, modelul SIR nu admite echilibru endemic cu $I^* > 0$. Aceasta reflectă faptul că, pe termen lung, epidemia se stinge întotdeauna (deși poate infecta o fracție mare din populație înainte de stingere).

Analiza stabilității locale. Deoarece $R = N - S - I$, sistemul se reduce la două ecuații (eliminând R):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dI}{dt} = \dfrac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.18}$$

Jacobianul sistemului (2.18) evaluat la un echilibru $(S^*, 0)$ este:

$$J(S^*, 0) = \begin{pmatrix} 0 & -\beta S^*/N \\ 0 & \beta S^*/N - \gamma \end{pmatrix}. \tag{2.19}$$

Valorile proprii sunt $\lambda_1 = 0$ și $\lambda_2 = \beta S^*/N - \gamma$.

Teoremă 2.12 (Stabilitatea echilibrului fără boală)**.** *Echilibrul fără boală* $(S^*, 0)$ *este:*

- *stabil (nelinear) dacă* $\beta S^*/N < \gamma$*, adică dacă* $S^*/N < 1/\mathcal{R}_0$*;*
- *instabil dacă* $\beta S^*/N > \gamma$*.*

Demonstrație. Valorea proprie $\lambda_2 = \beta S^*/N - \gamma$ determină stabilitatea pe direcția I. Deoarece $\lambda_1 = 0$, analiza liniară este degenerată și trebuie completată cu argumente neliniare. Totuși, din Teorema 2.10, $I(t)$ este descrescătoare dacă $S(t) < \gamma N/\beta$ pentru tot $t > 0$, ceea ce se întâmplă când $S_0 < \gamma N/\beta$, implicând stingerea epidemiei. □

2.3.4 Numărul de reproducere de bază $\mathcal{R}_0$

Definiție prin matricea generației următoare

Metoda *matricei generației următoare* (next-generation matrix), formalizată de Diekmann, Heesterbeek și Metz [4] și extinsă de van den Driessche și Watmough [5], furnizează o procedură sistematică pentru calculul $\mathcal{R}_0$ în modele compartimentale de complexitate arbitrară.

Fie $\mathbf{x}_I$ vectorul compartimentelor infectate. Scriem:

$$\frac{d\mathbf{x}_I}{dt} = \mathcal{F}(\mathbf{x}) - \mathcal{V}(\mathbf{x}), \tag{2.20}$$

unde $\mathcal{F}_i$ reprezintă rata de apariție a noilor infecții în compartimentul i, iar $\mathcal{V}_i$ reprezintă rata netă de ieșire din compartimentul i prin toate celelalte procese.

Matricele Jacobiene evaluate la echilibrul fără boală $\mathbf{x}_0$ sunt:

$$F = \left[\frac{\partial \mathcal{F}_i}{\partial x_j}\right]_{\mathbf{x}_0}, \quad V = \left[\frac{\partial \mathcal{V}_i}{\partial x_j}\right]_{\mathbf{x}_0}. \tag{2.21}$$

Matricea generației următoare este $K = FV^{-1}$, iar:

$$\mathcal{R}_0 = \rho(K), \tag{2.22}$$

unde $\rho(\cdot)$ este raza spectrală (cea mai mare valoare proprie în modul).

Calculul $\mathcal{R}_0$ pentru modelul SIR

Pentru modelul SIR, singurul compartiment infectat este $\{I\}$. La echilibrul fără boală $(S, I, R) = (N, 0, 0)$:

$$\mathcal{F}(I) = \frac{\beta SI}{N} \implies F = \left[\frac{\partial \mathcal{F}}{\partial I}\right]_{S=N} = \frac{\beta N}{N} = \beta. \tag{2.23}$$

$$\mathcal{V}(I) = \gamma I \implies V = \gamma. \tag{2.24}$$

Matricea K este scalarul:

$$K = FV^{-1} = \frac{\beta}{\gamma}. \tag{2.25}$$

Prin urmare:

$$\boxed{\mathcal{R}_0 = \frac{\beta}{\gamma}.} \tag{2.26}$$

Interpretarea biologică este directă: $1/\gamma$ este durata medie a perioadei infecțioase, iar β este numărul mediu de contacte eficiente pe unitate de timp; produsul lor dă numărul de infecții secundare produse pe durata infecțiozității.

Teoremă 2.13 (Pragul epidemic)**.** *Considerăm o populație inițial complet susceptibilă ($S_0 \approx N$). Atunci:*

(i) Dacă $\mathcal{R}_0 > 1$, numărul de infectați crește inițial exponențial și are loc o epidemie ($I(t)$ atinge un maxim $I_{\max} > I_0$).

(ii) Dacă $\mathcal{R}_0 \leq 1$, $I(t)$ este imediat descrescătoare și nu are loc nicio epidemie.

Demonstraţie. Din $dI/dt|_{t=0} = I_0(\beta S_0/N - \gamma) \approx I_0(\beta - \gamma)$ (pentru $S_0 \approx N$), concluzia urmează din semnul expresiei $\beta - \gamma$:

$$\left.\frac{dI}{dt}\right|_{t=0} > 0 \iff \beta > \gamma \iff \frac{\beta}{\gamma} > 1 \iff \mathcal{R}_0 > 1. \tag{2.27}$$

□

2.3.5 Pragul de imunizare în turmă (herd immunity)

Definiție 2.14 (Imunitate de grup)**.** Imunitatea de grup (*herd immunity*) se referă la protecția indirectă a persoanelor susceptibile, generată atunci când o fracție suficient de mare din populație este imunizată (fie natural, fie prin vaccinare), astfel încât transmiterea susținută a agentului patogen devine imposibilă.

Epidemia se stinge când numărul de reproducere efectiv devine subunitar:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N} < 1 \iff \frac{S(t)}{N} < \frac{1}{\mathcal{R}_0}. \tag{2.28}$$

Dacă o fracție p din populație este imunizată (cu $S/N = 1 - p$), condiția de blocare a epidemiei devine:

$$(1-p) \cdot \mathcal{R}_0 < 1 \iff p > 1 - \frac{1}{\mathcal{R}_0}. \tag{2.29}$$

Definiție 2.15 (Pragul de imunizare în turmă)**.** Fracția minimă a populației care trebuie imunizată pentru a preveni propagarea epidemiei este:

$$p_c = 1 - \frac{1}{\mathcal{R}_0}. \tag{2.30}$$

Exemplu 2.16. Pentru rujeolă, $\mathcal{R}_0 \approx 12$–18, deci $p_c \approx 92$–94%. Pentru COVID-19 (tulpina originală), $\mathcal{R}_0 \approx 2$–3, deci $p_c \approx 50$–67%. Variantele ulterioare (Delta, Omicron) au $\mathcal{R}_0$ semnificativ mai mare, crescând corespunzător pragul de imunizare.

2.3.6 Ecuația dimensiunii finale a epidemiei

O caracteristică remarcabilă a modelului SIR este că permite calculul analitic al *dimensiunii finale* a epidemiei — fracția totală din populație care este infectată în cursul epidemiei — fără a rezolva explicit sistemul diferențial.

Teoremă 2.17 (Ecuația dimensiunii finale)**.** *Fie $z = 1 - S(\infty)/N$ fracția din populație infectată cumulativ pe durata epidemiei, presupunând $S_0 = N$ și $R_0 = 0$. Atunci z este unica rădăcină pozitivă a ecuației transcendentale:*

$$z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.31}$$

Demonstrație. Împărțind ecuația pentru S la ecuația pentru R:

$$\frac{dS}{dR} = \frac{-\beta SI/N}{\gamma I} = -\frac{\beta}{\gamma N} S = -\frac{\mathcal{R}_0}{N} S. \tag{2.32}$$

Această ecuație separabilă în S și R se integrează direct:

$$\int_{S_0}^{S(t)} \frac{dS'}{S'} = -\frac{\mathcal{R}_0}{N} \int_0^{R(t)} dR' \implies \ln \frac{S(t)}{S_0} = -\frac{\mathcal{R}_0}{N} R(t). \tag{2.33}$$

Deci $S(t) = S_0 \exp(-\mathcal{R}_0 R(t)/N)$.

La $t \to \infty$, $I(\infty) = 0$ (demonstrat mai jos), deci $R(\infty) = N - S(\infty)$ (pentru

$R_0 = 0$). Substituind cu $S_0 = N$:

$$S(\infty) = Ne^{-\mathcal{R}_0(N - S(\infty))/N}. \tag{2.34}$$

Notând $z = 1 - S(\infty)/N$ și împărțind prin N:

$$1 - z = e^{-\mathcal{R}_0 z} \implies z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.35}$$

Că $I(\infty) = 0$: Funcția $R(t)$ este monoton crescătoare și mărginită de N, deci $\lim_{t\to\infty} R(t)$ există. Dacă $I(\infty) = \ell > 0$, atunci $R'(t) = \gamma I(t) \geq \gamma\ell > 0$ pentru t suficient de mare, ceea ce implică $R(t) \to \infty$, contradicție. Deci $I(\infty) = 0$.

Unicitatea rădăcinii pozitive: Fie $g(z) = 1 - e^{-\mathcal{R}_0 z} - z$. Avem $g(0) = 0$, $g'(0) = \mathcal{R}_0 - 1 > 0$ (dacă $\mathcal{R}_0 > 1$), și $g(1) = 1 - e^{-\mathcal{R}_0} - 1 = -e^{-\mathcal{R}_0} < 0$. Deoarece $g''(z) = -\mathcal{R}_0^2 e^{-\mathcal{R}_0 z} < 0$, funcția g este concavă, deci are exact o singură rădăcină în $(0, 1)$. □

Observație 2.18. Ecuația (2.31) este *transcendentală* și nu admite soluție în formă închisă pentru $\mathcal{R}_0 \neq 1$. Se rezolvă numeric, de exemplu prin metoda Newton-Raphson sau prin iterație de punct fix.

Corolar 2.19 (Rata de atac finală)**.** *Dacă $\mathcal{R}_0 > 1$, rata de atac finală z^* satisface $z^* \in (0, 1)$ și crește monoton cu $\mathcal{R}_0$. Limita $\lim_{\mathcal{R}_0\to\infty} z^* = 1$.*

2.3.7 Extensii ale modelului SIR

Modelul SIR de bază poate fi extins în multiple direcții pentru a acomoda realități biologice mai complexe:

- **Demografie:** includerea nașterilor și mortalității naturale permite studiul echilibrelor endemice pe termen lung.
- **Structura pe vârste:** matricele de contact structurate pe grupe de vârstă (modele de tip POLYMOD) capturează heterogenitatea contactelor sociale.
- **Spațialitate:** modelele cu patch-uri sau ecuații diferențiale parțiale descriu propagarea geografică.
- **Stochasticitate:** modelele stochastice (proces de ramificare, lanțuri Markov în timp continuu) sunt esențiale pentru populații mici.
- **Control:** vaccinarea, izolarea și tratamentul pot fi modelate ca fluxuri suplimentare între compartimente.

2.4 Metode Numerice pentru Ecuații Diferențiale Ordinare

2.4.1 Clasificarea metodelor numerice

Soluția analitică a sistemului SIR nu există în formă închisă (cu excepția cazurilor degenerate), ceea ce face indispensabilă rezolvarea numerică. O metodă numerică pentru problema cu valori inițiale (PVI):

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{2.36}$$

generează o secvență de aproximații $\mathbf{y}_n \approx \mathbf{y}(t_n)$ la punctele $t_n = t_0 + nh$.

Metodele numerice se clasifică după mai multe criterii:

- **Explicite vs. implicite:** la metodele explicite, $\mathbf{y}_{n+1}$ se calculează direct din valorile anterioare; la cele implicite, apare $\mathbf{y}_{n+1}$ și în membrul drept (sistem nonlinear de rezolvat la fiecare pas).
- **Un-pas vs. multi-pas:** metodele un-pas folosesc doar $\mathbf{y}_n$ pentru a calcula $\mathbf{y}_{n+1}$; metodele multi-pas folosesc mai mulți pași anteriori.
- **Ordin de convergență:** ordinul p al metodei determină că eroarea globală $\|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^p)$.
- **Pas fix vs. adaptiv:** metodele adaptive ajustează h dinamic pe baza unei estimări a erorii locale.

2.4.2 Metoda Euler explicită

Metoda Euler (1768) este cea mai simplă metodă numerică un-pas, obținută prin truncherea seriei Taylor la ordinul întâi:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n). \tag{2.37}$$

Teoremă 2.20 (Eroarea globală a metodei Euler)**.** *Dacă* $\mathbf{f}$ *este Lipschitz în* $\mathbf{y}$ *cu constanta* L *și* $\mathbf{y} \in C^2([t_0, T])$*, eroarea globală a metodei Euler satisface:*

$$\max_{n:t_n \le T} \|\mathbf{y}(t_n) - \mathbf{y}_n\| \le \frac{h}{2L} \max_{t \in [t_0,T]} \|\mathbf{y}''(t)\| \left(e^{L(T-t_0)} - 1\right) = \mathcal{O}(h). \tag{2.38}$$

Eroarea locală de trunchiere (pe un singur pas) este $\mathcal{O}(h^2)$, deci metoda are ordinul local 2 și ordinul global 1.

Aplicată sistemului SIR cu $\mathbf{y}_n = (S_n, I_n, R_n)^\intercal$, schema Euler este:

Algoritm 2.1 Metoda Euler pentru sistemul SIR

```
Input:  beta, gamma, N, S0, I0, R0, t0, tf, h
Output: Secventa (tn, Sn, In, Rn) pentru tn in [t0, tf]

(S, I, R) <- (S0, I0, R0);  t <- t0
while t < tf do
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    S  <- S + h * dS
    I  <- I + h * dI
    R  <- R + h * dR
    t  <- t + h
    Inregistreaza (t, S, I, R)
end while
```

Stabilitate. Analiza stabilității metodei Euler pe ecuația test $y' = \lambda y$ ($\lambda \in \mathbb{C}$) conduce la factorul de amplificare $z = 1 + h\lambda$. Metoda este stabilă dacă $|1 + h\lambda| \leq 1$, ceea ce definește domeniul de stabilitate (disc de rază 1 centrat în -1 în planul $h\lambda$). Această restricție limiteaza pasul de integrare h, în special pentru probleme *stiff*.

2.4.3 Metoda Runge-Kutta de ordinul 4 (RK4)

Metodele Runge-Kutta sunt metode un-pas explicite care realizează mai multe evaluări ale funcției $\mathbf{f}$ la fiecare pas, atingând ordine superioare fără a recurge la derivate de ordin înalt ale soluției.

Derivare și formulă

Metoda RK4 clasică (cu 4 stadii) este dată de:

$$\begin{aligned} \mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n), \\ \mathbf{k}_2 &= h \cdot \mathbf{f}\big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_1}{2}\big), \\ \mathbf{k}_3 &= h \cdot \mathbf{f}\big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_2}{2}\big), \\ \mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h,\ \mathbf{y}_n + \mathbf{k}_3), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \end{aligned} \tag{2.39}$$

Coeficienții metodei sunt reprezentați compact prin *tabloul Butcher*:

$$\begin{array}{c|cccc} 0 & & & & \\ 1/2 & 1/2 & & & \\ 1/2 & 0 & 1/2 & & \\ 1 & 0 & 0 & 1 & \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array} \tag{2.40}$$

Teoremă 2.21 (Ordinul metodei RK4)**.** *Metoda* (2.39) *are ordinul de convergență* $p = 4$*: dacă* $\mathbf{f} \in C^4$ *și* $\mathbf{y} \in C^5([t_0, T])$*, atunci eroarea globală satisface:*

$$\max_n \|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^4). \tag{2.41}$$

Schiță de demonstrație. Eroarea locală de trunchiere τ_n se obține substituind soluția exactă $\mathbf{y}(t_n)$ în schema (2.39) și expandând în serie Taylor. Se poate arăta că primele 4 condiții de ordin (condițiile de arbore Butcher pentru $p = 4$) sunt satisfăcute de coeficienții (2.40), implicând $\tau_n = \mathcal{O}(h^5)$. Eroarea globală este $\mathcal{O}(h^4)$ prin acumularea $\sim 1/h$ de pași. Detalii complete în [6]. □

Algoritm 2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR

```
Input:  f, y0, t0, tf, h
Output: Secventa (tn, yn)

y <- y0;  t <- t0
while t < tf do
    k1 <- h * f(t,        y           )
    k2 <- h * f(t + h/2, y + k1/2     )
    k3 <- h * f(t + h/2, y + k2/2     )
    k4 <- h * f(t + h,   y + k3       )
    y  <- y + (k1 + 2*k2 + 2*k3 + k4) / 6
    t  <- t + h
end while
```

2.4.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince)

Metodele cu pas fix impun utilizatorului alegerea unui h adecvat, care este o sarcină dificilă în practică. Metodele *adaptive* ajustează automat pasul h pe baza unei estimări a erorii locale, asigurând că eroarea rămâne sub o toleranță prestabilită ε.

Metode Runge-Kutta încorporate

O *pereche RK încorporată* de ordine p și $p+1$ calculează doi estimatori pentru $\mathbf{y}_{n+1}$ folosind *același* set de evaluări:

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} b_i \mathbf{k}_i \quad \text{(ordinul } p\text{)}, \tag{2.42}$$

$$\tilde{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} \tilde{b}_i \mathbf{k}_i \quad \text{(ordinul } p+1\text{)}, \tag{2.43}$$

cu $\mathbf{k}_i = \mathbf{f}(t_n + c_i h, \mathbf{y}_n + h\sum_{j<i} a_{ij}\mathbf{k}_j)$. Estimatorul de eroare locală este:

$$\mathbf{e}_{n+1} = \tilde{\mathbf{y}}_{n+1} - \hat{\mathbf{y}}_{n+1} = h\sum_{i=1}^{s}(b_i - \tilde{b}_i)\mathbf{k}_i. \tag{2.44}$$

Metoda Dormand-Prince (DOPRI5 / RK45)

Perechea **Dormand-Prince** [7], implementată ca `ode45` în MATLAB, `RK45` în SciPy (Python), `Tsit5` în Julia și `lsoda`/`ode45` în R (pachetul `deSolve`), utilizează 6 evaluări de funcție și are ordinul principal 5 cu estimator de eroare de ordinul 4.

Tabloul Butcher complet al metodei Dormand-Prince este:

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
$\hat{b}$	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\tilde{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

(2.45)

Control adaptiv al pasului

Pasul h este actualizat după fiecare pas pe baza normei erorii:

$$\text{err} = \left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} \cdot |\mathbf{y}_{n+1}|} \right\|_{\infty}, \tag{2.46}$$

unde atol și rtol sunt toleranțe absolute și relative. Pasul următor este calculat prin formula:

$$h_{\text{nou}} = h \cdot \min\big(f_{\max},\ \max\big(f_{\min},\ S \cdot \text{err}^{-1/(p+1)}\big)\big)\,, \tag{2.47}$$

cu factorul de siguranță $S = 0.9$ și limitele $f_{\min} = 0.1,\ f_{\max} = 10$.

2.4.5 Stabilitate numerică și probleme stiff

Definiție 2.22 (Problemă stiff)**.** O problemă cu valori inițiale (2.36) este numită *stiff* dacă valorile proprii ale Jacobianului $\partial\mathbf{f}/\partial\mathbf{y}$ au componente cu parte reală negativă de magnitudine mare, impunând pași de integrare h extrem de mici pentru metodele explicite din motive de stabilitate, chiar dacă soluția variază lent.

Aplicație la modelul SIR. Jacobianul sistemului SIR (2.18) este:

$$J(S, I) = \begin{pmatrix} -\beta I/N & -\beta S/N \\ \beta I/N & \beta S/N - \gamma \end{pmatrix}. \tag{2.48}$$

Valorile proprii depind de (S, I). La debutul epidemiei ($S \approx N$, I mic), valorile proprii sunt aproximativ 0 și $\beta - \gamma$, deci problema nu este stiff. Modelul SIR de bază nu este stiff pentru parametri tipici epidemiologici. Totuși, extensiile cu mai multe compartimente și rate foarte diferite pot produce probleme stiff, necesitând metode implicite (Runge-Kutta implicit, BDF).

Observație 2.23. Metodele implicite (de exemplu, metoda Euler implicită, Crank-Nicolson, Radau) au domenii de stabilitate nelimitate și sunt preferate pentru probleme stiff, în ciuda costului computațional superior (rezolvarea unui sistem nonlinear la fiecare pas).

2.5 Date Epidemiologice — Surse și Preprocesare

2.5.1 Surse de date epidemiologice

Implementarea și validarea modelelor epidemiologice computaționale necesită date empirice de calitate. Principalele surse de date epidemiologice utilizate în lucrare sunt:

- **WHO (Organizația Mondială a Sănătății):** rapoarte de situație, baze de date privind mortalitatea și morbiditatea, date de supraveghere a bolilor infecțioase.
- **ECDC (Centrul European pentru Prevenirea și Controlul Bolilor):** date de supraveghere pentru statele membre UE, inclusiv serii temporale zilnice pentru COVID-19 și alte boli infecțioase.

- **Johns Hopkins University CSSE:** setul de date COVID-19 (cazuri confirmate, decese, recuperați) la nivel de țară și regiune, disponibil public pe GitHub.
- **Our World in Data:** date armonizate din surse multiple, incluzând rate de vaccinare, indicatori de mobilitate și date socioeconomice.

2.5.2 Structura datelor și preprocesare

Datele epidemiologice brute prezintă probleme tipice care necesită preprocesare:

- **Valori lipsă:** interpolate liniar sau propagate din ultimul valor disponibil (forward fill);
- **Raportări retrospective:** corecții retroactive care produc valori negative în seriile de incidență zilnică;
- **Efecte de weekend:** ciclicitate săptămânală în raportare, uniformizată prin media mobilă de 7 zile;
- **Inconsistențe geografice:** modificări de granițe sau reorganizări administrative.

2.5.3 Estimarea parametrilor din date reale

Parametrii β și γ ai modelului SIR pot fi estimați din date observate prin metode statistice:

- **Cel mai mic pătrat neliniar (NLS):** minimizarea erorii pătratice dintre datele observate și soluția numerică a modelului;
- **Estimarea maximă a verosimilității (MLE):** presupunând un model de zgomot (Poisson sau binomial negativ pentru date de incidență), se maximizează funcția de log-verosimilitate;
- **Inferență bayesiană (MCMC):** permite cuantificarea incertitudinii parametrilor prin distribuții posterioare.

Estimatorul $\hat{\gamma}$ poate fi obținut direct dacă durata medie a perioadei infecțioase este disponibilă din studii clinice: $\hat{\gamma} = 1/T_{\text{infecțios}}$. Parametrul β este mai dificil de estimat și depinde de comportamentul social, intervenții și variante ale agentului patogen.

Capitolul 3

Tehnologii și Instrumente Utilizate

3.1 Limbaje de programare pentru calcul științific

Alegerea limbajului de programare pentru implementarea modelelor matematice nu este neutră: performanța, expresivitatea și ecosistemul de biblioteci au un impact direct asupra eficienței dezvoltării și calității rezultatelor. Această secțiune prezintă fundamentele și caracteristicile tehnice ale celor patru limbaje utilizate.

3.1.1 R

Istoric și filosofie

R este un limbaj de programare și mediu software pentru calcul statistic și grafică, descendent direct al limbajului S (Bell Laboratories, 1976). A fost creat de Ross Ihaka și Robert Gentleman la Universitatea din Auckland în 1993, iar în 1997 a fost adoptat ca proiect open-source de *R Development Core Team* [8]. Filosofia R este centrată pe date: vectorii, matricile și cadrele de date (data frames) sunt cetățeni de primă clasă, iar gramatica limbajului este proiectată pentru manipularea și analiza datelor.

Caracteristici tehnice relevante

R este un limbaj *interpretat*, dinamic tipizat, cu suport pentru programare funcțională, orientată pe obiecte (sistemele S3, S4 și R5/R6) și procedurală. Caracteristici relevante pentru calculul științific:

- **Vectorizare nativă:** operațiile aritmetice și logice aplicate pe vectori și matrice sunt automat vectorizate și executate prin cod C/Fortran compilat, evitând buclele interpretate costisitoare.
- **Lazy evaluation:** argumentele funcțiilor sunt evaluate doar când sunt necesare, permițând construcții expresive.

- **Gestionarea memoriei:** R utilizează un garbage collector cu copiere la scriere (copy-on-write), care poate genera overhead semnificativ pentru structuri de date mari.
- **Interfețe C/Fortran:** prin funcțiile `.C()`, `.Call()` și `.Fortran()`, R poate apela cod compilat nativ, stând la baza multor pachete de înaltă performanță.

Pachete relevante pentru modelul SIR

- **deSolve** [6]: pachetul principal pentru rezolvarea sistemelor de EDO, ecuații diferențiale cu derivate parțiale (EDP) și ecuații algebrico-diferențiale (EAD). Integrează solver-e consacrate: `lsoda` (Adams/BDF adaptiv), `ode45` (Dormand-Prince), `rk4` (Runge-Kutta fix). Interfața principală este funcția `ode()`.
- **ggplot2**: sistem de grafică bazat pe *grammar of graphics*, utilizat pentru vizualizarea traiectoriilor SIR.
- **data.table**: manipulare eficientă (în memorie) a seturilor de date mari, cu sintaxă concisă.
- **microbenchmark** și **bench**: măsurarea precisă a timpilor de execuție pentru benchmarking.

3.1.2 Julia

Istoric și filosofie

Julia a fost creat la MIT de Jeff Bezanson, Alan Edelman, Stefan Karpinski și Viral B. Shah, cu versiunea 1.0 lansată în august 2018 [9]. Motivația proiectului a fost depășirea *problemei celor două limbaje* (two-language problem): cercetătorii scriu prototipuri în R/Python (productivitate ridicată, viteză scăzută) și re-implementează codul critic în C/Fortran (viteză ridicată, productivitate scăzută). Julia vizează ambele obiective simultan.

Compilare Just-In-Time și sistemul de tipuri

Julia utilizează compilare **JIT** (Just-In-Time) bazată pe LLVM, cu o particularitate fundamentală: specializarea pe tipuri concrete. Când o funcție este apelată pentru prima dată cu argumente de tipuri specifice, compilatorul generează cod mașină optimizat pentru acele tipuri exacte. Apelurile ulterioare cu aceleași tipuri rulează direct codul compilat, apropiind performanța de C.

- **Multiple dispatch:** funcțiile pot fi definite cu comportamente diferite pentru combinații diferite de tipuri ale argumentelor (metode). Compilatorul selectează și compilează metoda corectă la momentul apelului.
- **Sistem de tipuri parametric:** tipurile pot fi parametrizate (ex. `Array{Float64, 1}`), permițând generarea eficientă de cod specializat.
- **Metaprogramming:** macrocomenzile Julia (`@macro`) permit generarea și transformarea codului la momentul parsării.

DifferentialEquations.jl

Ecosistemul **SciML** (Scientific Machine Learning) al Julia conține pachetul `DifferentialEquations.j` [10], cel mai cuprinzător framework open-source pentru rezolvarea ecuațiilor diferențiale. Acesta include:

- Peste 300 de algoritmi de rezolvare pentru EDO, EDP, EAD, ecuații diferențiale stochastice (EDS) și ecuații cu întârziere (DDEs);
- Solver-e de înaltă performanță: `Tsit5` (Tsitouras RK 4/5), `Vern7/9` (Verner 7/9), `Rodas4/5` (pentru probleme stiff);
- Integrare nativă cu GPU-uri și calcul distribuit;
- Diferențiere automată (AD) pentru estimarea parametrilor și identificarea sistemelor.

3.1.3 Python

Istoric și filosofie

Python a fost creat de Guido van Rossum și lansat în 1991 [11]. Filosofia sa, rezumată în *Zen of Python* (`PEP 20`), accentuează lizibilitatea și simplitatea codului. Python a devenit limbajul dominant în știința datelor și machine learning datorită ecosistemului său extins și comunității masive.

Caracteristici tehnice și ecosistem științific

Python este un limbaj interpretat, dinamic tipizat, multi-paradigmă. Principala limitare pentru calcul intensiv este **GIL** (Global Interpreter Lock), care previne execuția paralelă a thread-urilor Python pure. Performanța este recuperată prin biblioteci care rulează cod nativ C/Fortran în afara GIL:

- **NumPy**: calculul numeric vectorizat pe array-uri dense; operațiile sunt implementate în C/Fortran și nu sunt supuse GIL.

- **SciPy** [12]: algoritmică științifică, incluzând modulul `scipy.integrate` cu funcțiile `odeint` (wrapper LSODA) și `solve_ivp` (interfață modernă cu multiple metode: RK45, DOP853, Radau, BDF, LSODA).
- **Matplotlib**: vizualizare 2D/3D flexibilă.
- **Pandas**: manipularea datelor tabulare.

Funcțiile de rezolvare a EDO în SciPy

Interfața `solve_ivp` din `scipy.integrate` este apelată astfel:

```python
from scipy.integrate import solve_ivp
import numpy as np

def sir_model(t, y, beta, gamma, N):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt =  beta * S * I / N - gamma * I
    dRdt =  gamma * I
    return [dSdt, dIdt, dRdt]

# Parametri
N = 1_000_000; beta = 0.3; gamma = 0.1
S0, I0, R0 = N - 1, 1, 0
t_span = (0, 160); t_eval = np.linspace(0, 160, 1601)

sol = solve_ivp(
    fun=lambda t, y: sir_model(t, y, beta, gamma, N),
    t_span=t_span,
    y0=[S0, I0, R0],
    method='RK45',
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-10
)
```

Listing 3.1: Interfața solve_ivp din SciPy pentru rezolvarea sistemului SIR

3.1.4 SAS

Istoric și model de licențiere

SAS (Statistical Analysis System) a fost dezvoltat la North Carolina State University începând cu 1966 și comercializat de SAS Institute din 1976. Spre deosebire de R, Julia și Python, SAS este un software proprietar cu licențiere comercială, larg utilizat în industria farmaceutică, financiară și în cercetarea clinică, în parte datorită cerințelor reglementare (FDA validation, CFR 21 Part 11).

PROC IML pentru calcul numeric

PROC IML (Interactive Matrix Language) este procedura SAS care permite programare matriceală generală, analogă cu limbajele de tip MATLAB. Prin IML, sistemul SIR poate fi rezolvat implementând manual schema RK4:

```
proc iml;
  start sir_rhs(t, y, parms);
    beta = parms[1]; gamma = parms[2]; N = parms[3];
    S = y[1]; I = y[2]; R = y[3];
    dSdt = -beta * S * I / N;
    dIdt =  beta * S * I / N - gamma * I;
    dRdt =  gamma * I;
    return (dSdt || dIdt || dRdt);
  finish;

  start rk4_step(t, y, h, parms);
    k1 = h * sir_rhs(t,       y,           parms);
    k2 = h * sir_rhs(t + h/2, y + k1/2,   parms);
    k3 = h * sir_rhs(t + h/2, y + k2/2,   parms);
    k4 = h * sir_rhs(t + h,   y + k3,     parms);
    return (y + (k1 + 2*k2 + 2*k3 + k4) / 6);
  finish;

  /* Parametri si conditii initiale */
  N = 1000000; beta = 0.3; gamma = 0.1;
  parms = beta || gamma || N;
  y = (N-1) || 1 || 0;
  t = 0; h = 0.1; tf = 160;

  results = j(0, 4);
  do while (t <= tf);
    results = results // (t || y);
```

```
    y = rk4_step(t, y, h, parms);
    t = t + h;
  end;
quit;
```

Listing 3.2: Rezolvarea sistemului SIR în SAS/IML cu schema RK4

Comparație cu alternativele open-source

SAS prezintă avantaje în medii de producție reglementate (trasabilitate, audit trail, validare formală), dar dezavantaje semnificative pentru cercetare exploratorie: cost de licențiere ridicat, ciclu de actualizare lent și un ecosistem de pachete mult mai restrâns comparativ cu R și Python. Varianta **SAS OnDemand for Academics** oferă acces gratuit pentru uz educațional.

3.1.5 Analiză comparativă a limbajelor

Table 3.1 sintetizează principalele caracteristici ale celor patru limbaje din perspectiva utilizării pentru implementarea modelului SIR.

Tabela 3.1: Comparație a limbajelor de programare utilizate

Criteriu	**R**	**Julia**	**Python**	**SAS**
Licență	Open-source	Open-source	Open-source	Proprietar
Paradigmă	Funcț./OOP	Multi	Multi	Procedural
Compilare	Interpretat	JIT (LLVM)	Interpretat	Interpretat
Solver EDO	deSolve	DiffEq.jl	SciPy	PROC IML
Performanță rel.	Medie	Înaltă	Medie	Medie
Maturitate ecos.	Înaltă	Medie	Înaltă	Înaltă
Utilizare acad.	Ridicată	Medie	Ridicată	Ridicată

3.2 Baze de date distribuite — YugabyteDB

3.2.1 Concepte de baze de date distribuite

O *bază de date distribuită* este o colecție de date distribuite fizic pe mai multe noduri (mașini sau locații geografice), gestionată de un sistem software care oferă transparența localizării: aplicațiile interacționează cu baza de date ca și cum ar fi centralizată.

Definiție 3.1 (Teorema CAP)**.** Teorema CAP (Brewer, 2000) afirmă că un sistem distribuit nu poate garanta simultan mai mult de două dintre următoarele trei proprietăți:

- **Consistency (C):** toate nodurile văd aceleași date în același moment (citirile reflectă întotdeauna ultima scriere confirmată);
- **Availability (A):** orice cerere validă primește un răspuns (eventual fără ultima versiune a datelor);
- **Partition tolerance (P):** sistemul continuă să funcționeze chiar dacă comunicarea între noduri este parțial întreruptă.

Bazele de date distribuite moderne (YugabyteDB, CockroachDB, Spanner) optează pentru **CP** (consistency + partition tolerance), sacrificând disponibilitatea perfectă în favoarea coerenței datelor.

3.2.2 Arhitectura YugabyteDB

YugabyteDB [13] este o bază de date distribuită open-source, construită pentru a combina caracteristicile PostgreSQL cu scalabilitatea orizontală și toleranța la erori a sistemelor NoSQL. Arhitectura sa se bazează pe doi piloni principali:

1. Stratul de stocare DocDB. DocDB este motorul de stocare distribuit al YugabyteDB, bazat pe arhitectura **LSM-tree** (Log-Structured Merge-tree) derivată din **RocksDB**. Datele sunt stocate în *tablete* — unități de sharding distribuite pe noduri. Fiecare tabletă este replicată pe cel puțin trei noduri prin **protocolul Raft** (consens distribuit):

- Un nod este ales *lider*; scrierile trec prin lider;
- Leaderul replică modificarea pe nodurile urmașe (*followers*);
- O scriere este confirmată după ce o *majoritate* (quorum) de noduri o confirmă.

2. Nivelurile de interfață API.

- **YSQL** (YugabyteDB SQL): compatibil cu PostgreSQL 11+, suportând SQL complet, tranzacții ACID distribuite, join-uri, indecși, triggere și proceduri stocate;
- **YCQL** (YugabyteDB CQL): compatibil cu Apache Cassandra, optimizat pentru workload-uri de tip wide-column cu disponibilitate extinsă.

3.2.3 Sharding și replicare

YugabyteDB suportă două strategii de sharding:

- **Hash sharding:** cheia primară este hașurată și distribuită uniform pe tablete; asigură echilibru de sarcină dar nu suportă eficient *range scans*;

- **Range sharding:** tabletele acoperă intervale contigue de valori ale cheii; eficient pentru interogări de tip range, dar poate produce *hot spots*.

Observație 3.2. Pentru datele epidemiologice cu interogări frecvente pe intervale de date (serii temporale), range sharding pe coloana `timestamp` sau `date` este strategia recomandată, împreună cu un index secundar pe codul bolii și/sau regiune geografică.

3.3 Orchestrarea containerelor — Kubernetes

3.3.1 Containere și nevoia de orchestrare

Containerizarea (Docker) permite împachetarea unei aplicații împreună cu toate dependențele sale într-o unitate portabilă și reproductibilă. Pe măsură ce sistemele cresc în complexitate și numărul de containere depășește câteva zeci, gestionarea manuală devine infezabilă: unde rulează fiecare container? Cum se reporneşte dacă cade? Cum se scalează?

Kubernetes (K8s) [14, 15] este platforma open-source de orchestrare a containerelor, inițial dezvoltată la Google pe baza experienței cu sistemul intern Borg, și donată CNCF (Cloud Native Computing Foundation) în 2014.

3.3.2 Arhitectura Kubernetes

Un cluster Kubernetes este compus din:

Control Plane:

- **kube-apiserver:** componenta centrală, expune API-ul K8s (REST/gRPC); toate celelalte componente comunică exclusiv prin API server;
- **etcd:** baza de date distribuită key-value (Raft-based) care stochează starea întregului cluster;
- **kube-scheduler:** atribuie pod-uri nodurilor disponibile pe baza resurselor disponibile și a constrângerilor de afinitate;
- **kube-controller-manager:** rulează controllere care reconciliază starea dorită cu starea actuală (Deployment, ReplicaSet, StatefulSet etc.).

Worker Nodes:

- **kubelet:** agent pe fiecare nod, gestionează ciclul de viață al pod-urilor (Pull image, start/stop/restart containere);
- **kube-proxy:** gestionează regulile de rețea (iptables/IPVS) pentru accesul la servicii;

- **Container runtime:** interfața CRI (containerd, CRI-O).

3.3.3 Resurse Kubernetes relevante

- **Pod:** unitatea atomică de deployment; unul sau mai multe containere cu spațiu de rețea și stocare partajate;
- **Deployment:** gestionează un set de pod-uri identice; asigură restart automat și rolling updates;
- **StatefulSet:** similar Deployment, dar menține identitate stabilă pentru pod-uri (hostname, persistent volume); *esențial* pentru YugabyteDB;
- **Service:** abstractizare pentru accesul la un set de pod-uri; asigură load balancing și descoperire de servicii;
- **PersistentVolume / PersistentVolumeClaim:** stocare persistentă independentă de ciclul de viață al pod-ului;
- **ConfigMap / Secret:** configurare și date sensibile externalizate.

3.4 Backend API — Go

3.4.1 Caracteristici și filosofia limbajului Go

Go (cunoscut și ca Golang) este un limbaj de programare open-source creat la Google de Robert Griesemer, Rob Pike și Ken Thompson, cu prima versiune stabilă în 2012 [16]. Go a fost proiectat pentru productivitate la scară mare, cu accent pe simplitate, compilare rapidă și concurență structurată.

Caracteristici relevante pentru construcția unui API backend:

- **Compilare nativă rapidă:** Go compilează direct la cod mașin, producând binare statice fără dependențe externe;
- **Goroutine și canale (CSP):** concurența este modelată prin goroutine (fire de execuție ușoare, 2KB stack inițial) și canale (typed, pentru comunicare sincronă/asincronă); modelul Communicating Sequential Processes (CSP, Hoare 1978) este preferat față de memoria partajată cu mutex;
- **Interfețe structurale (duck typing):** un tip implementează implicit o interfață dacă definește metodele necesare, fără declarație explicită;
- **Garbage collection:** GC concurrent cu latență redusă ($<$ 1ms stop-the-world în versiunile recente);

- **Standard library bogată:** pachetul `net/http` suportă HTTP/1.1, HTTP/2 și TLS out-of-the-box, fără framework extern.

3.4.2 Ecosistem pentru API-uri REST

Deși `net/http` din biblioteca standard este suficient pentru API-uri simple, framework-uri ca **Gin** sau **Echo** adaugă:

- Router performant (radix tree pentru $\mathcal{O}(\log k)$ matching);
- Middleware pipeline (logging, CORS, rate limiting, autentificare JWT);
- Binding și validare automată a parametrilor HTTP (JSON, query params, path params);
- Generare automată a documentației OpenAPI/Swagger.

3.5 Frontend — Vue.js

3.5.1 Arhitectura Vue 3 și Composition API

Vue.js [17] este un framework JavaScript progresiv pentru construirea interfețelor utilizator, creat de Evan You în 2014. Vue 3 (2020) introduce **Composition API**, o alternativă la Options API care permite organizarea logicii prin funcționalitate (nu prin tip — data, methods, computed) și reutilizarea logicii prin *composables*.

Arhitectura Vue 3 se bazează pe:

- **Reactivity system:** bazat pe `Proxy` ES6, permite urmărirea dependențelor reactive cu granularitate fină;
- **Virtual DOM:** reconciliere eficientă a arborelui DOM cu algoritmul de diferențiere O(n);
- **Single File Components (SFC):** fișiere `.vue` care încapsulează template (HTML), script (JS/TS) și style (CSS) într-o unitate coezivă.

3.5.2 Ecosistem și biblioteci utilizate

- **Vue Router:** rutare client-side cu suport pentru rute dinamice, navigare programatică și guards de autentificare;
- **Pinia:** state management bazat pe Composition API, mai simplu decât Vuex și mai bine integrat cu TypeScript;

- **Axios:** client HTTP cu interceptori, pentru apelurile REST la API-ul Go;
- **ECharts / Chart.js:** vizualizare date epidemiologice (grafice SIR, comparații de performanță, hărți de densitate).

Capitolul 4

Arhitectura Sistemului E-Medicine

4.1 Vedere de ansamblu

Sistemul e-medicine propus în această lucrare este o aplicație distribuită, orientată pe microservicii, care permite utilizatorilor să încarce seturi de date epidemiologice reale, să declanșeze simulări ale modelului SIR în trei limbaje de programare distincte — R, Julia și Python — și să compare rezultatele atât din punct de vedere numeric, cât și al performanței computaționale. Arhitectura globală este organizată în patru straturi funcționale, ilustrate în Fig. 4.1.

Stratul de prezentare constă dintr-o aplicație web single-page (SPA) implementată în Vue.js, servită printr-un container Nginx. **Stratul API** este un server REST implementat în Go, responsabil cu autentificarea prin JWT, orchestrarea job-urilor de simulare și medierea accesului la baza de date. **Stratul de calcul** cuprinde trei microservicii dedicate — numite *runnere* — fiecare implementând algoritmul SIR într-un limbaj distinct; acestea comunică asincron cu API-ul printr-o coadă de mesaje Redis. **Stratul de stocare** este asigurat de un cluster YugabyteDB distribuit, care persistă atât datele epidemiologice de intrare, cât și rezultatele simulărilor. Întreaga infrastructură este orchestrată de Kubernetes, rulând pe un cluster cu trei noduri fizice, asigurând toleranță la erori, scalabilitate orizontală și management declarativ al resurselor.

Fluxul principal de date al sistemului se desfășoară după cum urmează. Utilizatorul se autentifică și obține un token JWT, după care poate încărca un fișier CSV cu date epidemiologice; API-ul validează structura fișierului, stochează metadatele în tabelul `datasets` și înregistrările brute în `epidemiological_records`. La inițierea unei simulări, utilizatorul specifică limbajul dorit și parametrii modelului (β, γ, N, $t_{\max}$); API-ul creează o înregistrare în `simulation_runs` cu statusul `pending` și publică un mesaj JSON în coada Redis corespunzătoare limbajului ales. Runner-ul consumă job-ul, execută simularea SIR, scrie seria temporală $(S(t), I(t), R(t))$ în `simulation_results`

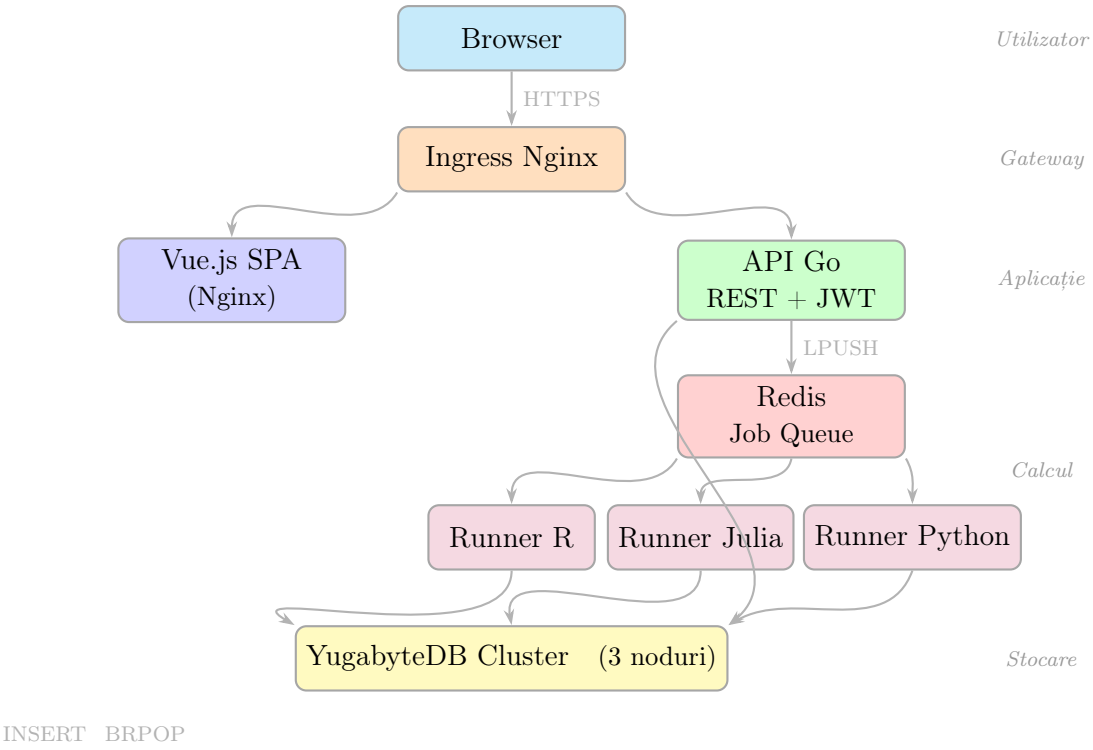


Figura 4.1: Arhitectura globală a sistemului e-medicine. Săgețile indică direcția de flux a datelor: browserul accesează aplicația prin Ingress, care rutează cererile către SPA sau API; API-ul publică job-uri în Redis și le consumă runner-ele dedicate; atât API-ul, cât și runner-ele accesează același cluster YugabyteDB.

și actualizează statusul la `completed`. Frontend-ul interoghează periodic API-ul și afișează graficele imediat ce simularea este finalizată.

4.2 Cerințe funcționale și non-funcționale

4.2.1 Cerințe funcționale

Prima cerință funcțională privește **autentificarea și autorizarea**: utilizatorii trebuie să se poată înregistra și autentifica, iar toate operațiile asupra datelor și simulărilor sunt protejate prin token JWT cu durată de valabilitate configurabilă.

Gestionarea seturilor de date epidemiologice presupune că utilizatorul poate încărca fișiere CSV ce conțin serii temporale epidemiologice — date calendaristice, numere de cazuri, populație, regiune geografică. Sistemul validează structura, calculează statistici sumare și stochează datele persistent în baza de date.

Configurarea și declanșarea simulărilor SIR permite utilizatorului să selecteze un set de date, să aleagă limbajul de execuție (R, Julia sau Python) și să introducă parametrii modelului (β, γ) și dimensiunea populației N. Sistemul execută simularea asincron, printr-un runner dedicat, și notifică finalizarea fără a bloca interfața.

Vizualizarea rezultatelor presupune afișarea curbelor $S(t)$, $I(t)$ și $R(t)$ ca grafice

interactive, cu posibilitatea de a suprapune valorile observate din datele de intrare peste curba simulată.

Compararea inter-limbaj permite utilizatorului să selecteze mai multe simulări efectuate pe același set de date, cu aceiași parametri, în limbaje diferite, și să compare vizual atât forma curbelor, cât și metricile de performanță (timp de execuție, memorie utilizată).

Exportul rezultatelor oferă posibilitatea de a descărca seria temporală a unei simulări în format CSV și de a exporta graficele ca imagini PNG.

Monitorizarea benchmark-urilor înseamnă că sistemul colectează automat timpii de execuție (wall-clock și CPU) și consumul de memorie pentru fiecare rulare și le expune printr-un endpoint dedicat, accesibil în interfața de comparare.

4.2.2 Cerințe non-funcționale

Din punct de vedere al **performanței**, latența endpointurilor API pentru operații de citire trebuie să fie sub 200 ms la percentila 95, în condiții normale de încărcare; runner-ele trebuie să finalizeze o simulare SIR standard ($N = 10^6$, $t_{\max} = 365$ zile) în mai puțin de 30 de secunde.

Disponibilitatea este asigurată de clusterul YugabyteDB cu factor de replicare 3, care tolerează căderea unui nod fără pierdere de date sau întrerupere a serviciului (disponibilitate $> 99{,}5\%$); API-ul Go este deploiat cu minimum două replici pentru eliminarea punctelor unice de eșec.

Scalabilitatea este obținută prin design-ul stateless al API-ului și prin arhitectura bazată pe coadă: numărul de replici ale oricărui runner poate fi crescut orizontal prin simpla modificare a câmpului `replicas` în Deployment-ul Kubernetes, fără modificări de cod; YugabyteDB suportă adăugarea de noduri fără downtime.

Securitatea implică protejarea tuturor endpointurilor sensibile prin JWT, gestionarea secretelor (chei JWT, parole DB) prin obiecte Kubernetes `Secret` — montate ca variabile de mediu, nu incluse în imaginile Docker — și aplicarea principiului privilegiilor minime la nivelul ServiceAccount-urilor Kubernetes.

Extensibilitatea garantează că adăugarea unui nou limbaj de implementare SIR necesită doar crearea unui nou container runner care respectă interfața de comunicare cu coada Redis, fără modificarea API-ului sau a schemei bazei de date.

Portabilitatea este asigurată prin containerizarea (Docker) a tuturor componentelor și descrierea lor declarativă prin manifeste Kubernetes, permițând deployment pe orice furnizor de cloud sau infrastructură on-premises care suportă Kubernetes 1.28+.

4.3 Arhitectura bazei de date distribuite

4.3.1 Schema relațională

Baza de date a sistemului este structurată în șase tabele principale, ale căror relații sunt ilustrate în Fig. 4.2.

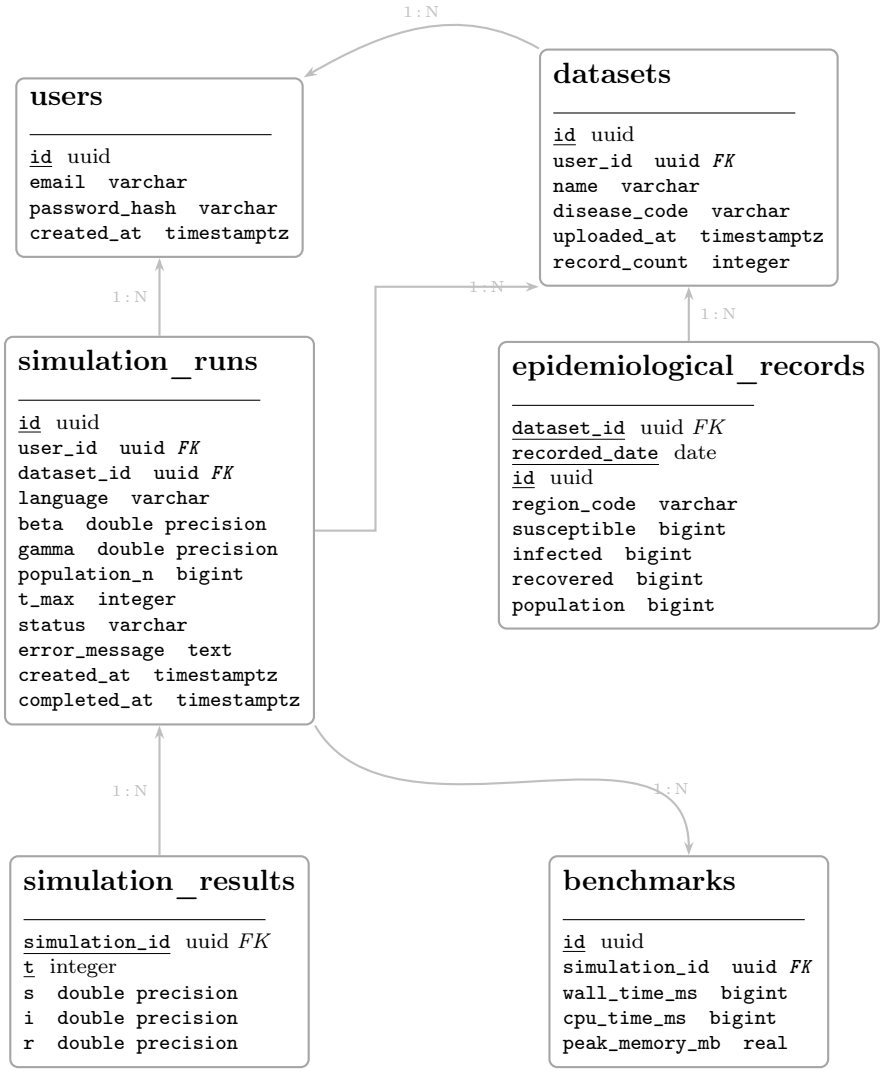


Figura 4.2: Diagrama entitate-relație a bazei de date. PK-urile sunt marcate cu aldine și subliniere; FK-urile sunt marcate cu *FK*; raportul de cardinalitate 1 : N este indicat pe fiecare relație.

Tabelul `users` stochează datele de autentificare ale utilizatorilor; parola nu este reținută în clar, ci ca hash bcrypt cu factor de cost 12. Tabelul `datasets` conține metadatele seturilor de date epidemiologice încărcate: autorul, codul bolii (ex. `COVID-19`, `SARS`), data încărcării și numărul total de înregistrări. Tabelul `epidemiological_records` stochează valorile brute ale seriei temporale; câmpurile `susceptible`, `infected` și `recovered` conțin valorile observate sau valorile inițiale folosite la calibrarea parametrilor, iar `region_code` urmează standardul ISO 3166-2. Tabelul `simulation_runs` reprez-

intă job-ul de simulare și include parametrii de intrare ai modelului SIR (β, γ, N, $t_{\max}$), limbajul ales, statusul execuției (`pending`, `running`, `completed`, `failed`), câmpul `error_message` pentru capturarea erorilor de execuție și timestampurile de creare și finalizare. Tabelul `simulation_results` stochează seria temporală de ieșire: valorile $s(t)$, $i(t)$ și $r(t)$ pentru fiecare pas de timp $t \in \{0, 1, \ldots, t_{\max}\}$; cheia primară compusă `(simulation_id, t)` identifică unic fiecare punct și asigură interogări eficiente ale seriei complete. Tabelul `benchmarks` reține metricile de performanță colectate automat la finalul fiecărei rulări: timpul wall-clock, timpul CPU și consumul maxim de memorie.

4.3.2 Modelul datelor epidemiologice

Datele epidemiologice sunt ingerate în sistem prin fișiere CSV cu structura minimă prezentată în Table 4.1.

Tabela 4.1: Structura minimă a fișierului CSV de date epidemiologice.

Câmp	**Tip**	**Descriere**
`date`	`DATE`	Data înregistrării (format ISO 8601, obligatoriu)
`region`	`VARCHAR`	Cod regiune ISO 3166-2 (obligatoriu)
`infected`	`BIGINT`	Număr de cazuri active (obligatoriu)
`population`	`BIGINT`	Populația totală (obligatoriu)
`recovered`	`BIGINT`	Cazuri recuperate (opțional)
`susceptible`	`BIGINT`	Populația susceptibilă estimată (opțional)

Câmpurile `susceptible` și `recovered` sunt opționale; dacă lipsesc, API-ul le estimează prin $S(0) = N - I(0)$ și $R(0) = 0$. Granularitatea temporală recomandată este zilnică; datele săptămânale sunt acceptate, dar reduc precizia estimării parametrilor. La încărcarea unui set de date, API-ul verifică că toate câmpurile obligatorii sunt prezente și au tipul corect, că relația $I(t) + R(t) \leq N$ este respectată pentru fiecare înregistrare, că seria temporală este ordonată cronologic și nu conține goluri mai mari de 14 zile consecutive (condiție tratată ca avertisment, nu eroare critică), și că fiecare cod de regiune este valid conform ISO 3166-2.

4.3.3 Strategia de partiționare și replicare în YugabyteDB

YugabyteDB utilizează un model de stocare bazat pe *tablets* (echivalentul shard-urilor), gestionate prin protocolul de consens Raft [18]. Clusterul este configurat cu factor de replicare $RF = 3$, garantând că pierderea oricărui nod unic nu afectează disponibilitatea sau consistența datelor.

Tabelul `epidemiological_records` este proiectat pentru *range sharding* pe cheia compusă `(dataset_id, recorded_date)`. Această strategie optimizează interogările de tip *range scan* pe interval temporal, care reprezintă cel mai frecvent pattern de acces — de exemplu, extragerea seriei temporale complete a unui set de date. În mediul de producție, partiționarea inițială se declară explicit prin clauza `SPLIT AT VALUES`, prezentată mai jos; migrația de bază aplică schema fără presplitting, lăsând YugabyteDB să partitioneze automat pe măsura creșterii datelor.

```
CREATE TABLE epidemiological_records (
    id              UUID DEFAULT gen_random_uuid(),
    dataset_id      UUID NOT NULL REFERENCES datasets(id),
    recorded_date   DATE NOT NULL,
    region_code     VARCHAR(10) NOT NULL,
    susceptible     BIGINT,
    infected        BIGINT NOT NULL,
    recovered       BIGINT,
    population      BIGINT NOT NULL,
    PRIMARY KEY (dataset_id, recorded_date, id)
) SPLIT AT VALUES (
    ('00000000-0000-0000-0000-000000000000', '2020-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2021-01-01'),
    ('00000000-0000-0000-0000-000000000000', '2022-01-01')
);
```

Listing 4.1: DDL de producție al tabelului `epidemiological_records` cu range sharding presplit pe cheie temporală.

Tabelul `simulation_results` utilizează, în schimb, hash sharding pe `simulation_id`, distribuind uniform rândurile între tablets, deoarece accesul la rezultate este întotdeauna filtrat după un `simulation_id` specific și nu necesită range scans inter-simulări.

Fiecare tablet are un lider Raft ales dintre cele trei replici; scrierile trec prin lider (*strong consistency*) și sunt replicate sincron pe celelalte două noduri înainte de confirmare. Parametrul `ysql_num_shards_per_tserver` este setat la 4 (față de valoarea implicită 8) pentru a reduce supraîncărcarea managementului de tablets pe un cluster de dimensiuni mici.

4.4 Arhitectura API-ului backend Go

4.4.1 Structura modulară a proiectului

Proiectul backend este organizat conform convențiilor comunității Go (*Standard Go Project Layout*) [19]:

```
emedicine-api/
+-- cmd/
|   `-- server/
|       `-- main.go        # Entry point: initializare, injectie dependinte
+-- internal/
|   +-- api/
|   |   +-- handlers/      # Handler HTTP per resursa (datasets, sims, auth)
|   |   +-- middleware/    # JWT validation, logging, CORS, rate-limit
|   |   `-- router/        # Configurare rute (chi router)
|   +-- db/
|   |   +-- migrations/    # Fisiere SQL versionate (golang-migrate)
|   |   `-- repository/    # Interfete si implementari repository pattern
|   +-- runner/
|   |   +-- dispatcher.go # Publicare job-uri in Redis
|   |   `-- queue.go       # Definitia mesajului JSON al job-ului
|   `-- models/            # Struct-uri de domeniu (User, Dataset, Run etc.)
`-- pkg/
    `-- sirmath/           # Implementare referinta SIR in Go (validare)
```

Listing 4.2: Structura de directoare a proiectului backend Go.

Arhitectura internă respectă pattern-ul în trei straturi. **Handler layer** parsează cererea HTTP, validează input-ul și serializează răspunsul JSON, fără a conține logică de business. **Service layer** orchestrează tranzacțiile, aplică regulile domeniului și publică job-uri în Redis. **Repository layer** abstractizează accesul la YugabyteDB prin interfețe Go, permițând testarea izolată a serviciilor fără a depinde de baza de date reală. Dependențele sunt injectate explicit prin constructori, conform idiomelor limbajului Go, fără un container IoC.

4.4.2 Mecanismul de autentificare JWT

Autentificarea se bazează pe JSON Web Tokens (RFC 7519) [20], utilizând algoritmul de semnătură `HS256` (HMAC-SHA256). Cheia secretă este generată la instalare și stocată ca Kubernetes Secret, montată în containerul API ca variabilă de mediu — nu

este inclusă în imaginea Docker. La autentificare, clientul trimite credențialele la `POST /api/auth/login`; API-ul verifică hash-ul bcrypt al parolei din baza de date și, în caz de succes, returnează un JWT cu payload `{sub: user_id, exp: now+24h, iat: now}`. Clientul include token-ul în fiecare cerere ulterioară prin header-ul `Authorization: Bearer <token>`, iar middleware-ul JWT îl validează — verificând semnătura și expierea — fără a accesa baza de date. Absența oricărei stări pe server este esențială pentru scalabilitatea orizontală: oricare replică a API-ului poate valida orice token, fără sincronizare între instanțe.

4.4.3 Diagrama endpointurilor REST

Table 4.2 prezintă endpointurile REST expuse de API.

Tabela 4.2: Endpointurile REST ale API-ului backend.

Metodă	**Cale**	**Descriere**	**Auth**
POST	`/api/auth/register`	Înregistrare utilizator nou	Nu
POST	`/api/auth/login`	Autentificare, returnează JWT	Nu
GET	`/api/datasets`	Lista seturilor de date ale utilizatorului	DA
POST	`/api/datasets`	Încărcare fișier CSV nou	DA
GET	`/api/datasets/{id}`	Detalii și statistici set de date	DA
DELETE	`/api/datasets/{id}`	Ștergere set de date	DA
POST	`/api/simulations`	Inițiere simulare nouă	DA
GET	`/api/simulations`	Lista simulărilor utilizatorului	DA
GET	`/api/simulations/{id}`	Status și metadate simulare	DA
GET	`/api/simulations/{id}/results`	Seria temporală $S(t)$, $I(t)$, $R(t)$	DA
GET	`/api/simulations/{id}/export`	Export CSV rezultate	DA
GET	`/api/benchmarks`	Benchmark-uri agregate per limbaj	DA
GET	`/api/health`	Health check (liveness probe K8s)	Nu

Răspunsurile API urmează o convenție simplificată inspirată din *JSON:API*: obiectul de nivel superior conține fie câmpul `data` (la succes), fie `error` cu un cod numeric și un mesaj uman-citibil (la eșec), permițând tratarea uniformă a erorilor în clientul frontend.

4.4.4 Integrarea cu runner-ele de algoritmi

Comunicarea dintre API-ul Go și runner-e este asincronă, mediată de Redis printr-un model *producer–consumer*. Când utilizatorul inițiază o simulare, API-ul publică un obiect JSON în lista Redis `sir:jobs:<language>` (ex. `sir:jobs:r`):

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id":    "7b4e1a02-...",
  "beta":          0.3,
  "gamma":         0.05,
  "population_n":  1000000,
  "t_max":         365,
  "db_dsn":        "postgres://emedicine:***@yb-tserver-
   service.yugabyte.svc.cluster.local:5433/emedicine"
}
```

Listing 4.3: Exemplu mesaj JSON publicat în coada Redis la inițierea unei simulări.

Fiecare runner execută o buclă `BRPOP` blocantă pe coada sa dedicată. La primirea unui job, runner-ul actualizează statusul simulării la `running`, execută algoritmul SIR (detaliat în Chapter 5), inserează rezultatele în `simulation_results` printr-un bulk insert de $t_{\max}+1$ rânduri (Python folosește `execute_values`, R folosește `dbWriteTable`, Julia folosește `LibPQ.load!`), înregistrează metricile de performanță în `benchmarks`, iar în final actualizează statusul la `completed` — sau la `failed`, cu mesajul de eroare stocat, dacă a apărut o excepție. Existența cozilor separate per limbaj (`sir:jobs:r`, `sir:jobs:julia`, `sir:jobs:python`) permite scalarea independentă a fiecărui runner în funcție de cerere, fără a modifica logica de rutare a API-ului.

4.5 Arhitectura frontend Vue.js

4.5.1 Structura componentelor

Frontend-ul este implementat ca o aplicație Vue.js 3 cu Composition API, folosind Vite ca bundler și TypeScript pentru siguranța tipurilor:

```
emedicine-ui/
‘-- src/
    +-- pages/
    |   +-- LoginPage.vue
    |   +-- RegisterPage.vue
    |   +-- DashboardPage.vue       # Sumar: nr. datasets,
   simulari recente
```

```
7     |   +-- DatasetsPage.vue        # Lista si upload datasets
8     |   +-- DatasetDetailPage.vue
9     |   +-- NewSimulationPage.vue
10    |   +-- SimulationResultPage.vue
11    |   +-- ComparePage.vue         # Comparare simulari inter-
      limbaj
12    |   `-- BenchmarksPage.vue
13    +-- components/
14    |   +-- SIRChart.vue            # Grafic Chart.js (S/I/R vs
      timp)
15    |   +-- BenchmarkTable.vue
16    |   +-- ParameterForm.vue       # Formular beta, gamma, N,
      t_max
17    |   +-- DatasetUpload.vue       # Drag-and-drop CSV
18    |   +-- SimulationStatus.vue    # Badge status cu polling
19    |   `-- NavBar.vue
20    +-- stores/
21    |   +-- auth.store.ts
22    |   +-- datasets.store.ts
23    |   +-- simulations.store.ts
24    |   `-- benchmarks.store.ts
25    +-- api/
26    |   `-- client.ts               # Axios instance cu JWT
      interceptor
27    `-- router/
28        `-- index.ts                # Vue Router cu navigation
      guards
```

Listing 4.4: Structura de directoare a proiectului Vue.js.

4.5.2 Fluxul de date și state management

Gestionarea stării globale este realizată cu Pinia [21], biblioteca oficială de state management pentru Vue.js 3. Fiecare store encapsulează starea, acțiunile (apeluri API asincrone) și getters (date derivate) pentru un domeniu specific.

La autentificare, `authStore` primește JWT-ul de la API și îl stochează în `localStorage` pentru persistență între sesiuni. Un interceptor Axios adaugă automat header-ul `Authorization: Bearer` la fiecare cerere ulterioară, iar navigation guards Vue Router redirecționează utilizatorii neautentificați spre pagina de login.

Fluxul de simulare începe din pagina `NewSimulationPage`, care apelează `simulationsStore.creat` cu parametrii formularului; store-ul primește înapoi `simulation_id` și componenta

`SimulationStatus` pornește un polling la interval de 2 secunde pe `GET /api/simulations/{id}`, oprindu-se când statusul devine `completed` sau `failed`. La finalizare, `SimulationResultPage` încarcă seria temporală și o pasează componentei `SIRChart`, care randează graficul interactiv cu trei curbe colorate distinct (S în albastru, I în roșu, R în verde).

Pagina `ComparePage` permite selectarea a 2–3 simulări efectuate pe același set de date. `simulationsStore.loadMultiple(ids)` execută cererile în paralel prin `Promise.all` și returnează seriile temporale comasate, pe care `SIRChart` le suprapune în același grafic, diferențiindu-le prin stil de linie (continuu, întrerupt, punctat).

4.6 Infrastructura Kubernetes

4.6.1 Topologia cluster-ului

Clusterul Kubernetes este compus din trei noduri: un nod de control plane și doi workeri, conform Fig. 4.3. Configurația hardware minimă recomandată este de 4 vCPU, 8 GiB RAM și 50 GiB SSD per nod, cu 8 vCPU și 16 GiB RAM recomandate pentru nodurile worker care găzduiesc runner-ele și instanțele YugabyteDB.

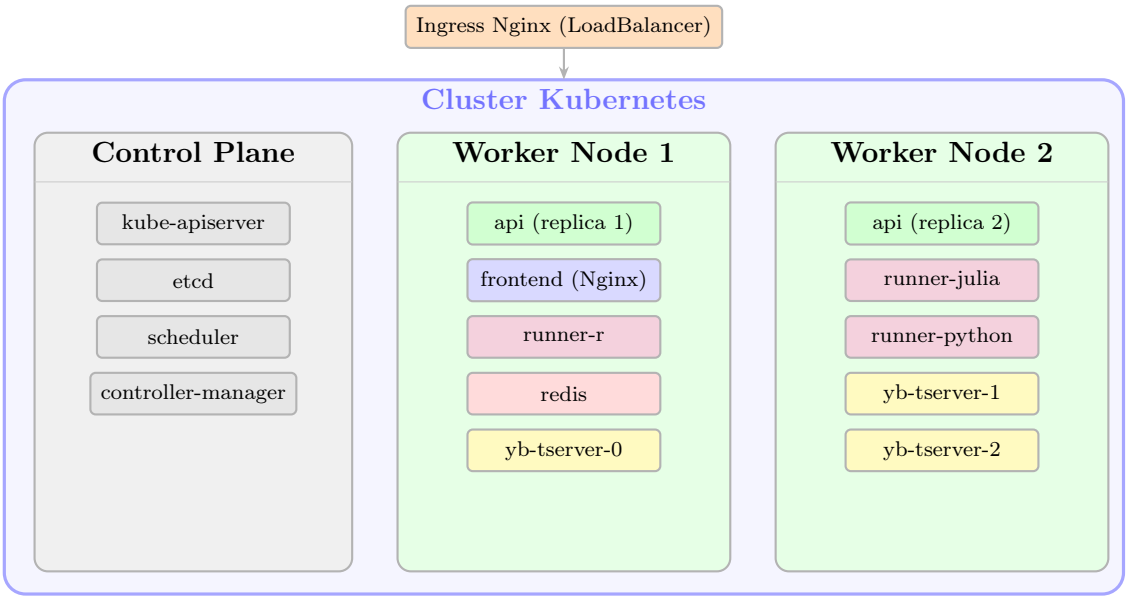


Figura 4.3: Topologia cluster-ului Kubernetes. Ingressul este expus ca LoadBalancer Service în fața clusterului. Worker Node 1 găzduiește frontend-ul, primul runner și prima instanță YugabyteDB tserver; Worker Node 2 găzduiește celelalte două runner-e și doi tserveri.

Toate resursele aplicației sunt grupate în namespace-ul `emedicine`, separat de namespace-ul `yugabyte` al bazei de date, facilitând aplicarea politicilor de rețea (NetworkPolicy) și controlul accesului bazat pe roluri (RBAC) la nivel de namespace.

4.6.2 Deployment YugabyteDB pe Kubernetes

YugabyteDB este deploiat prin Helm chart-ul oficial [22], configurând câte un StatefulSet cu 3 replici pentru componentele `yb-master` și `yb-tserver`:

```
helm repo add yugabytedb https://charts.yugabyte.com
helm install yugabyte yugabytedb/yugabyte \
  --namespace yugabyte --create-namespace \
  --set replicas.master=3 \
  --set replicas.tserver=3 \
  --set storage.master.size=10Gi \
  --set storage.tserver.size=50Gi \
  --set resource.tserver.requests.cpu=2 \
  --set resource.tserver.requests.memory=4Gi \
  --set gflags.tserver.ysql_enable_auth=true
```

Listing 4.5: Instalare YugabyteDB cu Helm.

Helm chart-ul creează automat StatefulSet-urile `yb-master` și `yb-tserver` cu câte 3 pod-uri fiecare, un `PersistentVolumeClaim` per pod (pe StorageClass `fast-ssd` dacă este disponibil, altfel `standard`), un headless Service pentru DNS intern stabil al pod-urilor (ex. `yb-tserver-0.yb-tservers.yugabyte.svc.cluster.local`), și un Service de tip ClusterIP pe portul 5433 (YSQL) utilizat de API-ul Go pentru conexiunea la baza de date.

4.6.3 Deployment API Go și frontend Vue.js

Imaginea Docker a API-ului este construită cu un *multi-stage build*: primul stage compilează binarul Go static (`CGO_ENABLED=0`), iar al doilea stage copiază doar binarul într-o imagine `distroless/static`, obținând o imagine finală de aproximativ 15 MiB. Deployment-ul Kubernetes configurează 2 replici, un liveness probe pe `GET /api/health` și un readiness probe care verifică conectivitatea la YugabyteDB înainte de a admite trafic.

Aplicația Vue.js este compilată în fișiere statice (`npm run build`) și servită printr-un container Nginx configurat ca SPA: toate rutele necunoscute sunt redirecționate la `index.html`, permițând Vue Router să gestioneze navigarea côté client.

Un singur Ingress resource (nginx-ingress controller) rutează traficul extern: cererile cu prefix `/api` sunt direcționate spre Service-ul `api-service` pe portul 8080, iar toate celelalte spre Service-ul `frontend-service` pe portul 80. În mediul de producție, terminarea TLS se configurează la nivel de Ingress prin cert-manager cu certificate Let's Encrypt; în mediul local (k3d), Ingress-ul este expus fără TLS.

Capitolul 5

Implementarea Modelului SIR

5.1 Seturile de date utilizate

5.1.1 Sursa datelor și formatul brut

Datele epidemiologice utilizate în această lucrare provin din setul de date *Our World in Data* (OWID) COVID-19 [23], un agregat public de date oficiale colectate de la autorități naționale de sănătate din întreaga lume și compilate zilnic de echipa OWID în colaborare cu Global Change Data Lab. Setul de date conține 429 435 de înregistrări zilnice acoperind 255 de locații (țări, continente, agregate globale) pentru perioada 5 ianuarie 2020 – 31 decembrie 2023, cu 67 de coloane per înregistrare. Coloanele relevante pentru modelul SIR sunt redate în Table 5.1.

Tabela 5.1: Coloanele OWID utilizate la construirea seturilor de date SIR.

Coloană OWID	**Tip**	**Rol în modelul SIR**
`iso_code`	`VARCHAR`	Cod regiune ISO 3166-1
`date`	`DATE`	Data înregistrării
`new_cases`	`BIGINT`	Cazuri noi zilnice
`total_cases`	`BIGINT`	Cazuri cumulate (proxy $R + I$)
`total_deaths`	`BIGINT`	Decese cumulate
`population`	`BIGINT`	Populația totală N

5.1.2 Preprocesarea și conversia în formatul sistemului

Formatul OWID nu conține direct compartimentele S, I, R ale modelului; acestea se estimează prin relațiile:

$$I(t) = \sum_{\tau=t-\Delta}^{t} \text{new_cases}(\tau), \tag{5.1}$$

$$R(t) = \text{total_cases}(t) - I(t) - \text{total_deaths}(t), \tag{5.2}$$

$$S(t) = N - I(t) - R(t), \tag{5.3}$$

unde $\Delta = 14$ zile reprezintă fereastra de infecțiozitate estimată pentru COVID-19. Valorile inițiale $S(0)$, $I(0)$, $R(0)$ pentru fiecare scenariul de simulare se extrag din prima zi a perioadei de interes.

Scriptul de preprocesare filtrează rândurile cu `new_cases < 0` (corecții retroactive frecvente în datele OWID), interpolează liniar golurile de maxim 7 zile consecutive și elimină perioadele cu mai mult de 14 zile lipsă. Rezultatul este exportat în fișiere CSV cu structura descrisă în Table 4.1 din capitolul 4, gata de ingestie prin `POST /api/datasets`.

5.1.3 Parametrii estimați și scenariile de simulare

Parametrii β și γ ai modelului SIR se estimează prin minimizarea erorii pătratice medii (MSE) dintre seria temporală observată $\hat{I}(t)$ și soluția numerică $I(t;\, \beta, \gamma)$:

$$(\hat{\beta}, \hat{\gamma}) = \arg\min_{\beta,\gamma} \sum_{t=0}^{T} \left[\hat{I}(t) - I(t;\, \beta, \gamma)\right]^2, \tag{5.4}$$

rezolvată cu metoda Nelder-Mead [24] implementată în `optim()` (R) și `scipy.optimize.minimize()` (Python).

Table 5.2 prezintă parametrii estimați pentru cele trei scenarii folosite în evaluarea performanței din capitolul 6.

Tabela 5.2: Parametrii SIR estimați pentru scenariile de simulare.

Scenariu	**Regiune**	**Perioadă**	N	β	γ	R_0	$t_{\max}$
COVID-19 val. 1	România (RO)	Mar–Iun 2020	19 237 691	0,1413	0,0714	1,98	121
COVID-19 val. 4	România (RO)	Sep–Dec 2021	19 100 000	0,1082	0,0714	1,51	121
Referință SIR	Sintetic	—	10 000 000	0,3000	0,0500	6,00	365

Parametrul γ este fixat la $1/14 \approx 0{,}0714\,\text{zi}^{-1}$ pentru ambele scenarii COVID-19, corespunzând unei perioade infecțioase medii de 14 zile documentată clinic [**?**]; β este estimat prin minimizare Nelder-Mead pe cazurile cumulate $C(t) = I(t) + R(t)$.

$R_0 \approx 1{,}98$ pentru valul 1 și $R_0 \approx 1{,}51$ pentru valul 4 reflectă transmisibilitatea efectivă în prezența intervențiilor non-farmaceutice (lockdown, distanțare socială), nu valoarea de bază $R_0^{\text{bază}} \approx 4$–$6$ specific COVID-19 fără restricții.

Scenariul *Referință SIR* cu $N = 10^7$ și $t_{\max} = 365$ zile este folosit ca benchmark computațional standard, deoarece generează exact $t_{\max} + 1 = 366$ de puncte de ieșire, independent de datele reale.

5.2 Implementarea în R

5.2.1 Structura proiectului și dependențe

Runner-ul R este un script autonom (`runner.R`) cu dependențele gestionate la build-time prin `install_packages.R`:

```r
install.packages(
  c("deSolve", "DBI", "RPostgres", "redux", "jsonlite"),
  repos = "https://cloud.r-project.org", quiet = TRUE
)
```

Listing 5.1: Dependențele runner-ului R.

Rolul fiecărui pachet este:

- **deSolve** [25] — rezolvitor de ODE/PDE/DAE pentru R, interfața principală de integrare numerică;
- **DBI** / **RPostgres** — abstracție și driver PostgreSQL pentru scrierea bulk a rezultatelor în YugabyteDB;
- **redux** — client Redis bazat pe `hiredis` (C), utilizat pentru bucla `BRPOP` de consum al job-urilor;
- **jsonlite** — serializare/deserializare JSON pentru parsarea mesajelor din coadă.

5.2.2 Definiția sistemului ODE și metoda RK4

Sistemul diferențial SIR este definit ca o funcție R compatibilă cu interfața `deSolve::ode()`:

```r
sir_model <- function(time, state, parameters) {
  with(as.list(c(state, parameters)), {
    N  <- S + I + R
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    list(c(dS, dI, dR))
```

```
  })
}

# Conditii initiale si parametri
state      <- c(S = N - 1, I = 1, R = 0)
parameters <- c(beta = beta, gamma = gamma)
times      <- seq(0, t_max, by = 1)

out <- ode(y = state, times = times,
           func = sir_model, parms = parameters,
           method = "rk4")
```

Listing 5.2: Definiția sistemului SIR în R și apelul `ode()`.

Metoda `"rk4"` corespunde metodei Runge-Kutta clasice de ordinul 4 cu pas fix $h = 1$ zi. Alegerea unui pas fix este justificată de natura problemei: seria temporală discretă zilnică face inutilă controlul adaptiv al pasului, iar RK4 oferă eroare de trunchiere $O(h^5)$ per pas (eroare globală $O(h^4)$), suficientă pentru precizia cerută de benchmark. Pachetul `deSolve` suportă și metodele adaptive `lsoda` și `ode45`, dar acestea introduc overhead suplimentar al controlului erorii fără beneficii de acuratețe la $h = 1$.

5.2.3 Colectarea metricilor și stocarea în baza de date

Timpul de execuție se măsoară cu `proc.time()`, care returnează atât timpul wall-clock (`elapsed`) cât și timpii CPU user și system. Consumul de memorie se estimează prin `gc(verbose=FALSE)`, care forțează un ciclu de garbage collection și returnează memoria heap utilizată de R în acel moment:

```
wall_start <- proc.time()["elapsed"]
cpu_start  <- proc.time()["user.self"]

out <- ode(y = state, times = times, func = sir_model,
           parms = parameters, method = "rk4")

wall_ms <- as.integer((proc.time()["elapsed"] - wall_start) *
   1000)
cpu_ms  <- as.integer((proc.time()["user.self"] - cpu_start) *
    1000)
mem_mb  <- as.numeric(gc(verbose=FALSE)[2,2])

# Inserare bulk via dbWriteTable (COPY intern)
df <- as.data.frame(out)
colnames(df) <- c("t", "s", "i", "r")
```

```
df$simulation_id <- sim_id
df$t <- as.integer(df$t)
dbWriteTable(con, "simulation_results",
             df[, c("simulation_id","t","s","i","r")],
             append = TRUE, row.names = FALSE)
```

Listing 5.3: Colectarea metricilor și inserarea bulk în YugabyteDB.

Funcția `dbWriteTable(append=TRUE)` din pachetul `RPostgres` generează intern o comandă `COPY FROM STDIN` în format binar, evitând parsarea rând-cu-rând specifică INSERT-urilor individuale. Aceasta este metoda cu cel mai scăzut overhead pentru inserarea seriilor temporale de $t_{\max} + 1$ puncte.

5.3 Implementarea în Julia

5.3.1 Structura proiectului și pachete

Runner-ul Julia este organizat ca un proiect Julia standard cu `Project.toml`. Pachetele sunt instalate la build-time prin `Pkg.add`:

```
RUN julia --project=. -e \
  'using Pkg; Pkg.add(["DifferentialEquations","JSON3","LibPQ
   ","Redis"])'
```

Listing 5.4: Instalarea dependențelor în imaginea Docker Julia.

Rolul pachetelor:

- **DifferentialEquations.jl** [10] — ecosistemul unificat pentru ecuații diferențiale al lui Julia; include peste 300 de solver-e;
- **JSON3.jl** — serializare JSON performantă, folosind inferența de tipuri din Julia pentru evitarea alocărilor;
- **LibPQ.jl** — client PostgreSQL nativ Julia cu suport pentru protocolul binar și inserare bulk prin `LibPQ.load!`;
- **Redis.jl** — client Redis cu interfață blocantă `brpop`.

5.3.2 ODEProblem și solver-ul Tsit5

Definiția sistemului SIR respectă interfața *in-place* a `DifferentialEquations.jl`, indicată de convenția `!` (funcția modifică vectorul derivatelor `du` în loc să aloce unul nou):

```
function sir_ode!(du, u, p, t)
    S, I, R = u
    beta, gamma = p
    N = S + I + R
    du[1] = -beta * S * I / N         # dS/dt
    du[2] =  beta * S * I / N - gamma * I   # dI/dt
    du[3] =  gamma * I                # dR/dt
end

u0    = [N - 1.0, 1.0, 0.0]
tspan = (0.0, Float64(t_max))
t_save = Float64.(0:t_max)
prob  = ODEProblem(sir_ode!, u0, tspan, (beta, gamma))

sol = solve(prob, Tsit5(),
            saveat  = t_save,
            reltol  = 1e-6,
            abstol  = 1e-8)
```

Listing 5.5: Definiția sistemului SIR în Julia și apelul `solve()`.

Solver-ul `Tsit5()` [26] este o metodă Runge-Kutta explicită de ordinul 5(4) cu control adaptiv al pasului (pereche Dormand-Prince de tip "free interpolant"). Față de RK4 cu pas fix, Tsit5 ajustează automat h astfel încât eroarea locală relativă să rămână sub `reltol` $= 10^{-6}$, garantând o soluție mai precisă pentru sisteme cu dinamică variabilă (tranziții rapide în faza de creștere exponențială a epidemiei). Parametrul `saveat` forțează salvarea soluției exact la punctele întregi $t \in \{0, 1, \ldots, t_{\max}\}$, fără a afecta pașii adaptivi interni.

5.3.3 Colectarea metricilor și inserarea bulk

```
wall_start = time_ns()
sol = solve(prob, Tsit5(), saveat=t_save, reltol=1e-6, abstol
   =1e-8)
wall_ms = div(time_ns() - wall_start, 1_000_000)

# Bulk insert prin LibPQ.load!
npts = length(sol.t)
LibPQ.load!(
    (simulation_id = fill(sim_id, npts),
     t = Int.(sol.t),
```

```
10      s = getindex.(sol.u, 1),
11      i = getindex.(sol.u, 2),
12      r = getindex.(sol.u, 3)),
13     conn,
14     "INSERT INTO simulation_results(simulation_id,t,s,i,r)
15      VALUES(\$1,\$2,\$3,\$4,\$5)"
16 )
```

Listing 5.6: Benchmarking și inserare bulk în Julia.

Funcția `time_ns()` returnează timpul monoton în nanosecunde, asigurând rezoluție ridicată fără overhead. `LibPQ.load!` transmite datele prin protocolul binar PostgreSQL în mod batch, similar `COPY`, evitând round-trip-urile individuale per rând.

5.4 Implementarea în Python

5.4.1 Structura proiectului și dependențe

Dependențele runner-ului Python sunt declarate în `requirements.txt`:

```
1 scipy>=1.13.0
2 psycopg2-binary>=2.9.9
3 redis>=5.0.4
```

Listing 5.7: `requirements.txt` al runner-ului Python.

- **scipy** — biblioteca de calcul științific Python; oferă `scipy.integrate.solve_ivp` cu multiple metode de integrare ODE;
- **psycopg2-binary** — driver PostgreSQL pentru Python, cu suport pentru `execute_values` (batch INSERT);
- **redis** — client Redis Python pentru bucla `brpop`.

5.4.2 scipy.integrate.solve_ivp cu metoda RK45

```
1 def sir_ode(t, y, beta, gamma):
2     S, I, R = y
3     N = S + I + R
4     dS = -beta * S * I / N
5     dI =  beta * S * I / N - gamma * I
6     dR =  gamma * I
7     return [dS, dI, dR]
8
```

```
y0     = [N - 1, 1, 0]
t_span = (0, t_max)
t_eval = list(range(t_max + 1))

sol = solve_ivp(
    sir_ode, t_span, y0, args=(beta, gamma),
    method  = "RK45",
    t_eval  = t_eval,
    rtol    = 1e-6,
    atol    = 1e-8
)
```

Listing 5.8: Definiția sistemului SIR în Python și apelul `solve_ivp()`.

Metoda `"RK45"` din `scipy.integrate.solve_ivp` implementează metoda Dormand-Prince de ordinul 5(4) cu control adaptiv al erorii [27], echivalentul Python al `Tsit5()` din Julia. Toleranțele `rtol`=10^{-6}, `atol`=10^{-8} sunt alese consistent cu cele din Julia pentru comparabilitatea rezultatelor numerice.

5.4.3 Colectarea metricilor și inserarea bulk

```
import time, resource
import psycopg2.extras

wall_start = time.perf_counter()
cpu_start  = time.process_time()

sol = solve_ivp(sir_ode, t_span, y0, args=(beta, gamma),
                method="RK45", t_eval=t_eval,
                rtol=1e-6, atol=1e-8)

wall_ms = int((time.perf_counter() - wall_start) * 1000)
cpu_ms  = int((time.process_time() - cpu_start)  * 1000)
mem_mb  = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss /
    1024.0

# Bulk insert prin execute_values
rows = [
    (sim_id, int(t), float(sol.y[0][i]),
             float(sol.y[1][i]), float(sol.y[2][i]))
    for i, t in enumerate(sol.t)
]
```

```
psycopg2.extras.execute_values(
    cur,
    "INSERT INTO simulation_results"
    "(simulation_id, t, s, i, r) VALUES %s",
    rows
)
```

Listing 5.9: Benchmarking și inserare bulk în Python.

Python oferă două surse independente de timp CPU: `time.perf_counter()` măsoară timpul wall-clock monoton cu rezoluție sub-microsecundă, iar `time.process_time()` măsoară suma timpilor user+system ai procesului, eliminând timpii de așteptare I/O. Consumul de memorie este citit din structura `rusage` a kernel-ului Linux (`ru_maxrss` = peak RSS în KiB). Funcția `psycopg2.extras.execute_values` transmite toate rândurile într-un singur mesaj SQL prin template-uri cu valori multiple, reducând semnificativ latența față de INSERT-uri individuale.

5.5 Implementarea de referință în SAS/IML

5.5.1 RK4 manual în PROC IML

SAS/IML (Interactive Matrix Language) [28] nu include solver-e ODE adaptive native; integrarea numerică se implementează manual folosind operații matriciale. Runner-ul SAS nu face parte din sistemul containerizat curent — motivat de absența unei imagini Docker oficiale SAS/IML cu licență Open Source — dar implementarea de referință prezentată mai jos documentează comportamentul echivalent și servește drept bază de comparație pentru analiza acurateței numerice din capitolul 6.

```
proc iml;
  /* Parametri */
  beta  = 0.30;  gamma = 0.071;
  N     = 19237691;
  t_max = 365;   h = 1;

  /* Conditii initiale: S, I, R */
  y = {19237690, 1, 0};
  results = j(t_max+1, 4, 0);
  results[1,] = (0 || y`);

  do t = 1 to t_max;
    S = y[1]; I = y[2]; R = y[3];

```

```
    /* Pas RK4 */
    k1 = h * sir_rhs(y, beta, gamma, N);
    k2 = h * sir_rhs(y + 0.5*k1, beta, gamma, N);
    k3 = h * sir_rhs(y + 0.5*k2, beta, gamma, N);
    k4 = h * sir_rhs(y + k3, beta, gamma, N);
    y  = y + (k1 + 2*k2 + 2*k3 + k4) / 6;

    results[t+1,] = (t || y‘);
  end;

  /* Export rezultate */
  create sir_output from results
    [colname={"t","S","I","R"}];
  append from results;
  close sir_output;
quit;

start sir_rhs(y, beta, gamma, N);
  S = y[1]; I = y[2]; R = y[3];
  dS = -beta * S * I / N;
  dI =  beta * S * I / N - gamma * I;
  dR =  gamma * I;
  return (dS // dI // dR);
finish;
```

Listing 5.10: Implementare RK4 manual în SAS/IML pentru modelul SIR.

5.5.2 Considerații privind precizia și limitările SAS/IML

Implementarea RK4 cu pas fix $h = 1$ în SAS/IML produce rezultate numerice identice cu runner-ul R (care folosește tot RK4, $h = 1$), cu diferențe la nivelul erorilor de rotunjire floating-point. Principala limitare a abordării SAS este absența metodelor adaptive native: pentru sisteme cu dinamică rigidă (stiff ODEs) sau cu parametri care implică separări mari de scară temporală, pasul fix poate acumula erori semnificative sau impune un h nejustificat de mic pentru stabilitate numerică.

Din perspectiva integrării în sistem, SAS/IML poate fi apelat ca subprocess dintr-un container Docker bazat pe SAS Viya sau SAS 9, cu comunicare prin fișiere intermediare sau prin conectori JDBC la YugabyteDB. Această abordare nu a fost implementată în versiunea curentă a sistemului, dar arhitectura bazată pe cozi Redis permite adăugarea unui runner SAS fără modificarea API-ului sau a schemei bazei de date.

5.6 Interfața comună de integrare cu API-ul Go

5.6.1 Protocolul de comunicare prin cozi Redis

Spre deosebire de abordările clasice bazate pe subprocese (comunicare stdin/stdout), sistemul utilizează un model *producer-consumer* asincron mediat de Redis. Acest design decuplează complet API-ul de runner-e: API-ul nu cunoaște starea sau localizarea runner-elor, iar runner-ele nu cunosc API-ul — singura interfață comună este schema mesajului JSON din coadă.

Ciclul de viață al unui job de simulare este prezentat în Fig. 5.1.

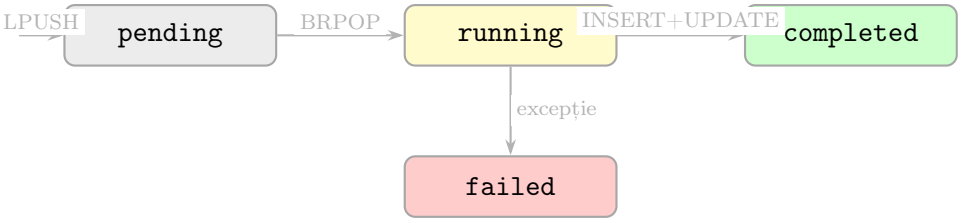


Figura 5.1: Ciclul de viață al unui job de simulare: tranziția de stare în `simulation_runs` în funcție de evenimentele din runner.

5.6.2 Schema mesajului de job

API-ul publică un obiect JSON în lista Redis `sir:jobs:<language>` (unde `<language>` ∈ {`r`, `julia`, `python`}) prin `LPUSH`, cu structura:

```
{
  "simulation_id": "a3f9c2d1-...",
  "dataset_id":    "7b4e1a02-...",
  "beta":          0.30,
  "gamma":         0.071,
  "population_n":  19237691,
  "t_max":         365,
  "db_dsn":        "postgres://emedicine:***@yb-tserver-
   service
                    .yugabyte.svc.cluster.local:5433/emedicine
   "
}
```

Listing 5.11: Structura mesajului JSON de job publicat de API în coada Redis.

Câmpul `db_dsn` permite runner-ului să se conecteze direct la YugabyteDB fără a cunoaște în avans adresa bazei de date — adresa este injectată la momentul creării job-ului de către API, din variabila de mediu `DATABASE_URL` a containerului API.

5.6.3 Bucla de consum și tratarea erorilor

Fiecare runner implementează o buclă infinită de consum blocant:

```
while True:
    try:
        result = rdb.brpop(queue, timeout=5)
        if result is None:
            continue          # timeout fara job, retry
        _, data = result
        job = json.loads(data)
        run_simulation(job)
    except Exception as e:
        print(f"queue error: {e}", file=sys.stderr)
        time.sleep(2)         # back-off la eroare Redis
```

Listing 5.12: Bucla generică de consum Redis (ilustrată în Python).

`BRPOP` cu `timeout=5` blochează conexiunea la Redis cel mult 5 secunde; dacă nu sosește niciun job în această perioadă, returnează `nil` și bucla iterează imediat. Această abordare menține o singură conexiune TCP persistentă la Redis, evitând overhead-ul reconectărilor frecvente. În cazul unei erori Redis (pierdere conexiune, restart), runner-ul introduce un back-off de 2 secunde înainte de reîncercare, evitând spiralele de erori rapide.

Excepțiile apărute în interiorul funcției `run_simulation` sunt capturate explicit, iar statusul simulării este actualizat la `failed` cu mesajul de eroare stocat în coloana `error_message` a tabelului `simulation_runs`, permițând diagnosticarea post-mortem din interfața web.

Capitolul 6

Evaluarea Performanței și Scalabilității

6.1 Metodologia de benchmarking

6.1.1 Configurații hardware utilizate

Tabel cu configurațiile hardware: laptop (Intel i5, 8GB RAM), workstation (AMD Ryzen 9, 32GB RAM), server cloud (AWS EC2 c5.4xlarge). Sistemul de operare, versiunile limbajelor, versiunile solver-elor.

6.1.2 Metrici urmărite

Metrici: (1) Timp de execuție wall-clock (mediana și IQR peste 30 rulări), (2) CPU time (user + system), (3) Peak memory usage (RSS), (4) Acuratețe numerică (eroarea față de soluția de referință DOPRI8 cu toleranță 10^{-12}).

6.1.3 Protocoale de testare și reproductibilitate

Protocol: warm-up runs, izolare procese, seed fix, versioning al codului de benchmark. Toate configurațiile rulate de 30 de ori, cu colectarea automatizată a rezultatelor.

6.2 Scalabilitate în funcție de dimensiunea setului de date

Experimente cu $N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$. Grafice log-log ale timpului de execuție față de N pentru fiecare limbaj. Identificarea complexității empirice.

6.3 Comparație timp de execuție — R vs Julia vs Python vs SAS

Tabel comparativ și grafice boxplot. Analiza statistică: test Kruskal-Wallis, comparații post-hoc Dunn pentru detectarea diferențelor semnificative. Discutarea overhead-ului de start-up al fiecărui limbaj față de timpul de calcul pur.

6.4 Consum de memorie și resurse

Profile de memorie pentru fiecare implementare. Grafice de utilizare a memoriei în timp. Identificarea cauzelor alocărilor excesive.

6.5 Acuratețea numerică a rezultatelor

Compararea soluțiilor numerice cu soluția de referință. Eroarea L^∞ și L^2 pentru $S(t)$, $I(t)$, $R(t)$. Verificarea conservării $S + I + R = N$ (eroarea de conservare).

6.6 Performanța bazei de date distribuite

6.6.1 Latență la scriere și citire

Măsurarea latenței P50/P95/P99 pentru operații INSERT și SELECT pe tabelele epidemiologice. Comparație cu PostgreSQL standalone.

6.6.2 Comportament sub load — teste de stress

Teste de load cu `pgbench` și/sau `cassandra-stress`. Throughput (tranzacții/secundă) față de numărul de clienți concurenți. Comportamentul clusterului la oprirea unuia din cele 3 noduri YugabyteDB.

6.7 Performanța API-ului Go

Benchmarking API cu `wrk` sau `hey`: RPS (requests/second), latență P95/P99, comportament sub concurență (10, 50, 100, 500 clienți simultani).

6.8 Analiză comparativă sintetică

Tabel de scoruri normalizate per metrică și per limbaj. Radar chart (spider chart) cu performanța relativă. Recomandări pentru scenarii de utilizare: volum mic vs. mare, precizie vs. viteză.

Capitolul 7

Aplicația Web E-Medicine

7.1 Descrierea funcționalităților aplicației

Prezentare generală a aplicației: ecranele principale, fluxurile de utilizare cheie. Screenshot-uri ale interfeței finale.

7.2 Backend Go — Implementare detaliată

7.2.1 Structura proiectului Go

Arborele de directoare al proiectului Go, convenții de denumire, separarea responsabilităților pe pachete: api, db, runner, models, config.

7.2.2 Implementarea endpointurilor REST

Codul Go pentru endpointurile principale, utilizând framework-ul ales (Gin/Echo). Demonstrarea pattern-ului: handler -> service -> repository -> DB.

7.2.3 Gestionarea execuției algoritmilor

Implementarea runner-elor asincrone: pool de goroutine, channel-uri pentru rezultate, context cancellation pentru timeout-uri. Codul Go pentru lansarea subprocess-urilor R, Julia, Python, SAS.

7.2.4 Conexiunea la YugabyteDB

Driverul `pgx` pentru PostgreSQL/YSQL, connection pooling, query-uri parametrizate pentru prevenirea SQL injection.

7.2.5 Autentificare și securitate

JWT middleware, CORS configuration, rate limiting, validarea input-ului, HTTPS cu certificate TLS.

7.3 Frontend Vue.js — Implementare detaliată

7.3.1 Structura componentelor și pagini

Arborele de componente, structura directorului `src/`, tipare de compunere (composables reutilizabile).

7.3.2 Dashboard epidemiologic

Componenta Dashboard: metrici cheie (R0 curent, vârf de infectați, dimensiunea finală estimată), grafice de sumar, ultimele rulări.

7.3.3 Vizualizarea graficelor SIR

Componenta SIRChart cu ECharts/Chart.js: afișarea traiectoriilor S(t), I(t), R(t), zoom interactiv, export PNG/SVG. Codul Vue 3 (Composition API).

7.3.4 Comparatorul de rezultate între limbaje

Pagina de comparare: suprapunerea graficelor din rulări multiple (R, Julia, Python, SAS pe același set de date), tabelul de diferențe numerice, graficul timpilor de execuție.

7.3.5 Interfața pentru seturile de date

CRUD pentru seturile de date: upload CSV, previzualizare, editarea metadatelor, ștergere. Validarea client-side a formularelor.

7.4 Fluxuri de utilizare (use cases)

Descrierea pas-cu-pas și screenshot-uri pentru: (1) Încărcarea unui set de date nou, (2) Declanșarea și monitorizarea unei simulări, (3) Compararea rezultatelor între limbaje, (4) Exportul rezultatelor.

7.5 Testare și validare funcțională

Unit teste Go (testing package), integration tests (testcontainers), end-to-end tests (Playwright/Cypress pentru Vue.js). Acoperire de cod.

Capitolul 8

Concluzii și Direcții de Dezvoltare Viitoare

8.1 Sinteza contribuțiilor

Rezumarea contribuțiilor principale ale lucrării: implementările comparative, infrastructura distribuită, aplicația web. Ce a fost atins față de obiectivele din Capitolul 1.

8.2 Concluzii privind implementările modelului SIR

Care limbaj s-a dovedit cel mai performant și în ce condiții? Care oferă cel mai bun compromis performanță/ușurință de utilizare? Recomandări pentru practicieni.

8.3 Concluzii privind infrastructura distribuită

Evaluarea YugabyteDB + K8s pentru date epidemiologice: avantaje (scalabilitate, fault tolerance), dezavantaje (complexitate operațională). Potrivirea arhitecturii pentru sistemele e-medicine.

8.4 Limitări ale lucrării

Limitări recunoscute: modelul SIR de bază ignoră structura de vârstă, spațialitatea și stochasticitatea; benchmarkurile sunt limitate la o serie de configurații hardware; interfața web nu implementează autentificare completă în versiunea prezentată.

8.5 Direcții de dezvoltare viitoare

Direcții: (1) Extinderea la modele SEIR, SIRS, modele stochastice; (2) Integrarea de modele de machine learning pentru estimarea parametrilor; (3) Adăugarea de vizualizări spațiale (hărți de propagare); (4) Suport pentru date în timp real (streaming cu Apache Kafka); (5) Implementarea unui modul de alertare epidemiologică automată; (6) Extinderea benchmarkurilor la GPU (CUDA, ROCm) pentru Julia și Python.

Bibliografie

[1] Roy M. Anderson and Robert M. May. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press, Oxford, 1991.

[2] Herbert W. Hethcote. The mathematics of infectious diseases. *SIAM Review*, 42(4):599–653, 2000.

[3] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[4] Odo Diekmann, Johan A. P. Heesterbeek, and Johan A. J. Metz. On the definition and the computation of the basic reproduction ratio $\mathcal{R}_0$ in models for infectious diseases in heterogeneous populations. *Journal of Mathematical Biology*, 28(4):365–382, 1990.

[5] Pauline van den Driessche and James Watmough. Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission. *Mathematical Biosciences*, 180(1–2):29–48, 2002.

[6] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.

[7] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[8] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.

[9] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[10] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.

[11] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[12] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[13] Yugabyte, Inc. YugabyteDB architecture documentation. Documentație tehnică, 2024.

[14] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[15] The Kubernetes Authors. Kubernetes documentation. Documentație tehnică, 2024.

[16] The Go Authors. The Go programming language specification. Specificație oficială, 2024.

[17] Evan You et al. Vue.js documentation. Documentație oficială, 2024.

[18] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC'14)*, pages 305–319, Philadelphia, PA, 2014. USENIX Association.

[19] Rakyll and contributors. Standard Go project layout. Depozit GitHub, 2023.

[20] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT). Request for Comments, Internet Engineering Task Force, 2015.

[21] Evan You and contributors. Pinia — the intuitive store for Vue.js. Documentație oficială, 2024.

[22] Yugabyte, Inc. YugabyteDB Helm chart for Kubernetes. Depozit GitHub, 2024.

[23] Edouard Mathieu, Hannah Ritchie, Lucas Rodés-Guirao, Cameron Appel, Charlie Giattino, Joe Hasell, Bobbie Macdonald, Saloni Dattani, Diana Beltekian, Esteban Ortiz-Ospina, and Max Roser. Coronavirus pandemic (COVID-19). Our World in Data, 2020.

[24] J. A. Nelder and R. Mead. A simplex method for function minimization. *The Computer Journal*, 7(4):308–313, 1965.

[25] Karline Soetaert, Thomas Petzoldt, and R. Woodrow Setzer. Solving differential equations in R: Package deSolve. *Journal of Statistical Software*, 33(9):1–25, 2010.

[26] Ch. Tsitouras. Runge-Kutta pairs of order 5(4) satisfying only the first column simplifying assumption. *Computers & Mathematics with Applications*, 62(2):770–775, 2011.

[27] J. R. Dormand and P. J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[28] SAS Institute Inc. *SAS/IML 15.2 User's Guide*. SAS Institute Inc., Cary, NC, 2020.

Anexa A

Cod Sursă — Implementarea R

Codul R complet pentru modelul SIR: definire sistem EDO, apel deSolve, estimare parametri, export JSON. Inclus prin `\lstinputlisting{...}` din fișierele sursă.

Anexa B

Cod Sursă — Implementarea Julia

Codul Julia complet: ODEProblem, DifferentialEquations.jl, benchmarking cu BenchmarkTools.jl, export JSON3.

Anexa C

Cod Sursă — Implementarea Python

Codul Python complet: scipy.integrate.solve_ivp, implementare manuală RK4 NumPy, benchmarking cu timeit, export JSON.

Anexa D

Cod Sursă — Implementarea SAS

Codul SAS/IML complet: PROC IML cu RK4 manual, PROC EXPORT, rutine de benchmarking.

Anexa E

Schema Bazei de Date

Diagrama ERD completă și scripturile SQL de creare a tabelelor (CREATE TABLE, indecși, constrângeri). Incluse ca listing-uri SQL.

Anexa F

Configurații Kubernetes

Manifestele YAML complete: StatefulSet YugabyteDB, Deployment API Go, Deployment frontend Nginx, Services, PVCs, ConfigMaps, Secrets, Ingress.

Anexa G

Documentația API REST

Specificația OpenAPI 3.0 (YAML) a API-ului Go sau echivalentul documentat manual: fiecare endpoint cu metoda HTTP, URL, parametrii, corpul cererii (request body), răspunsuri posibile (2xx, 4xx, 5xx) și exemple JSON.